

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energíá–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase, cuyo aporte metodológico central es un procedimiento reproducible para etiquetar ventanas de ejecución como *compute-bound* o *memory-bound* mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy–Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	10
Planteamiento y justificación del problema	11
Objetivos	13
1. Marco de referencia	15
1.1. Marco Conceptual	15
1.1.1. Termodinámica del silicio y potencia CMOS	15
1.1.2. Modelo Roofline y regímenes de limitación del rendimiento	16
1.1.3. Telemetría microarquitectónica	18
1.1.4. Espacios de ejecución: <i>user-space</i> y <i>kernel-space</i>	19
1.1.5. Interfaces estándar de observabilidad e instrumentación	20
1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y <i>governors</i>	21
1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks	23
1.1.8. Aprendizaje automático supervisado ligero	23
1.1.9. Producto Energía–Retardo (EDP)	24
1.2. Marco Legal	25
1.3. Estado del Arte	25
2. Metodología	28
2.1. Enfoque metodológico	28
2.2. Diseño metodológico	29

2.2.1.	Población y muestra	29
2.2.2.	Plataforma experimental	29
2.2.3.	Instrumentos de recolección	30
2.2.4.	Variables observadas	31
2.3.	Fase 1: Recolección de información y caracterización de cargas	33
2.3.1.	Verificación previa de la plataforma	33
2.3.2.	Catálogo de cargas de trabajo	33
2.3.3.	Calibración y criterio de etiquetado	34
2.3.4.	Caracterización de intensidad operacional y calibración Roofline en GPU	35
2.3.5.	Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional	38
2.3.6.	Matriz experimental y control de sesgos	39
2.3.7.	Post-procesamiento y control de calidad de los datos	40
2.3.8.	Medición directa de FLOPs por ventana y su validación empírica . .	43
2.3.9.	Reproducibilidad	46
2.4.	Fase 2: Desarrollo del modelo de aprendizaje automático	47
2.5.	Fase 3: Implementación del agente de control DVFS	47
2.6.	Fase 4: Validación experimental y análisis de resultados	48
2.7.	Aspectos éticos	49
3.	Resultados	50
3.1.	Descripción del conjunto de datos experimental	50
3.2.	Validación de la actuación DVFS en CPU	51
3.3.	Caracterización de precisión aritmética en el catálogo de GPU	52
3.4.	Estado del control de frecuencia en GPU	54
3.5.	Medición directa de bytes movidos en memoria (<i>uncore</i>)	55
3.5.1.	Interferencia del propio instrumento sobre la medición que reemplaza	57
3.6.	Validación cruzada de los techos Roofline con una herramienta independiente	59
4.	Discusión	62
5.	Conclusiones	66
5.1.	Limitaciones	67
5.2.	Trabajo Futuro	67
	Repositorio y recursos complementarios	73

Índice de tablas

2.1. Características de la plataforma experimental.	30
2.2. Señales de telemetría capturadas por ventana de muestreo.	32
2.3. Composición del catálogo experimental de cargas de trabajo.	34
2.4. Estados de frecuencia de la matriz experimental.	39
2.5. Taxonomía de anotaciones de calidad por ventana de muestreo.	43
2.6. FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.	44

Índice de figuras

1.1. Modelo Roofline conceptual: relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Fuente: tomada de Williams et al. [7, Fig. 1(a)].	16
---	----

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como `ondemand` o `schedutil`, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente

en espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como *compute-bound* o *memory-bound* a partir de telemetría microarquitectónica. Con base en esta clasificación, el sistema ajusta dinámicamente la frecuencia de CPU y GPU con el objetivo de optimizar el Producto Energía–Retardo (EDP). Se espera que este enfoque permita mejorar la eficiencia energética en comparación con los gobernadores tradicionales de Linux, manteniendo una degradación controlada del rendimiento y un *overhead* mínimo asociado a la inferencia.

Ahora bien, antes de que un modelo de aprendizaje automático pueda decidir sobre la frecuencia de operación, es necesario disponer de un conjunto de datos etiquetado que asocie observaciones de telemetría con regímenes de ejecución. Construir ese conjunto no es un paso preparatorio trivial: exige resolver de forma explícita cómo se atribuye la telemetría al proceso medido, con qué resolución temporal se observa, bajo qué criterio se declara que una ventana de ejecución pertenece a un régimen u otro, y qué observaciones deben descartarse por no ser representativas. Estas decisiones condicionan la validez de todo lo que se construya después, razón por la cual el presente documento dedica una parte sustancial de su desarrollo metodológico a la construcción y verificación del instrumento de medición.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [1] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [2].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-*

bound, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 4].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [5], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [1, 2, 3, 4, 5]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS) en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del

modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje para la optimización energética se fundamenta en la relación no lineal entre los parámetros de operación del hardware y la disipación de calor. A continuación, se definen los principios termodinámicos, arquitectónicos y de aprendizaje computacional que rigen el diseño de este agente de control en espacio de usuario, así como los conceptos de instrumentación y medición que sustentan la construcción del conjunto de datos experimental.

1.1.1. Termodinámica del silicio y potencia CMOS

La disipación de potencia en un circuito CMOS moderno se descompone en potencia estática (corrientes de fuga) y potencia dinámica, originada por la conmutación de cargas capacitivas. Como DVFS actúa sobre frecuencia y voltaje, su efecto principal recae sobre la potencia dinámica, modelable para una compuerta como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α es el factor de actividad, C la capacitancia efectiva conmutada y V , f el voltaje y la frecuencia de operación [6]. Dado que operar a mayor frecuencia suele exigir mayor voltaje por integridad temporal, reducir frecuencia habitualmente permite también reducir voltaje, con una caída de potencia más que proporcional por la dependencia cuadrática en V . Este acoplamiento es la base física del compromiso energía–rendimiento en DVFS: cuánto se gana en potencia al bajar el punto operativo depende del comportamiento computacional

predominante de la aplicación, no solo de la reducción de frecuencia en sí.

1.1.2. Modelo Roofline y regímenes de limitación del rendimiento

El modelo Roofline analiza el rendimiento de una aplicación en función de su intensidad operacional y de los límites físicos del hardware:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} es el rendimiento aritmético pico, I la intensidad operacional (operaciones por byte transferido desde memoria) y BW_{pico} el ancho de banda pico [7]. Si el término activo es $I \cdot BW_{\text{pico}}$ el kernel es *memory-bound*; si es P_{pico} , es *compute-bound*.

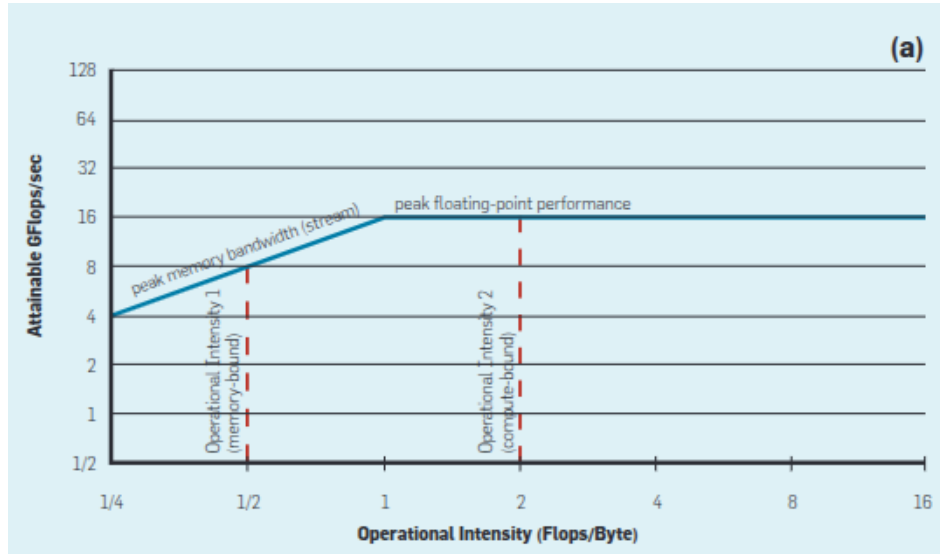


Figura 1.1: Modelo Roofline conceptual: relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Fuente: tomada de Williams et al. [7, Fig. 1(a)].

Punto de inflexión y calibración empírica del techo

La frontera entre regímenes es el punto de inflexión o *ridge point*, donde ambas ramas de (1.2) se igualan:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

En el criterio operacional adoptado en este trabajo, una ventana con intensidad observa-

da por debajo de I_{ridge} se etiqueta como *memory-bound*; por encima, como *compute-bound* [7]. Cuando el dispositivo admite precisiones aritméticas con rendimientos pico sustancialmente distintos —como una GPU con rutas de simple y doble precisión—, un único I_{ridge} genérico no es representativo de ninguna de las dos y la ecuación debe evaluarse por separado para cada precisión (sección 2.3.4).

Los valores nominales de fábrica de P_{pico} y BW_{pico} describen máximos teóricos que pueden no reproducir las condiciones de la configuración experimental concreta, como el número de núcleos activos, la anchura vectorial o los canales de memoria poblados [34]. Por ello, en este trabajo ambos techos se determinan mediante calibración empírica: cargas diseñadas para saturar cada límite por separado —ancho de banda de memoria y densidad aritmética sobre datos residentes en caché—, ejecutadas bajo la misma afinidad de núcleos que la campaña experimental. I_{ridge} resulta así una propiedad de la plataforma bajo la configuración utilizada, no una constante tomada directamente de la ficha técnica.

Estimación de la intensidad operacional observada

Aplicar el criterio anterior a una ventana concreta exige estimar:

$$I = \frac{\text{FLOPs realizados en la ventana}}{\text{bytes movidos en la ventana}} \quad (1.4)$$

El numerador no es una magnitud trivialmente observable: la disponibilidad y semántica de los eventos PMU asociados a aritmética de punto flotante no son uniformes entre microarquitecturas, y un evento puede contabilizar instrucciones retiradas, micro-operaciones ejecutadas o actividad especulativa según el procesador, una fuente de sobreconteo ya documentada empíricamente [35]. En este trabajo, ningún evento PMU se considera válido por defecto: su *encoding* se determina para la microarquitectura concreta y su interpretación se contrasta mediante microbenchmarks de resultado conocido. El procedimiento completo: *encoding* exacto, ponderación por ancho vectorial y tipo de operación, y validación empírica con error relativo por debajo de 0.5 % contra un kernel de trabajo conocido analíticamente se documenta en la sección 2.3.8, junto con la campaña real que confirmó su disponibilidad sostenida a escala.

El denominador es más difícil de observar directamente: el tráfico real hacia memoria principal se origina en el controlador de memoria, mientras que los contadores accesibles por proceso observan desde la perspectiva del núcleo, sin visibilidad directa sobre ese tráfico. Este trabajo lo mide mediante los propios contadores del controlador de memoria (*uncore*), cuyas condiciones de acceso se discuten en la sección 1.1.3 y cuyo resultado de medición se reporta

en la sección 3.5.

1.1.3. Telemetría microarquitectónica

Los procesadores modernos incorporan Unidades de Monitorización de Rendimiento (PMU), compuestas por contadores fijos (eventos frecuentes como ciclos e instrucciones) y programables (eventos configurables), que permiten instrumentación de baja sobrecarga sobre el flujo de instrucciones y la jerarquía de memoria [8]. El número de contadores físicos disponibles simultáneamente es limitado — típicamente 1–4 fijos y 4–8 genéricos en arquitecturas modernas [8] —, lo que condiciona qué eventos pueden observarse a la vez.

Multiplexación de contadores y escalado de medidas

Cuando los eventos solicitados exceden los contadores físicos disponibles, el kernel rota periódicamente los eventos sobre ellos (multiplexación) y escala cada lectura mediante la razón entre tiempo habilitado y tiempo efectivamente contando. Este escalado es una extrapolación — asume tasa de evento uniforme durante los intervalos no observados —, supuesto que se debilita cuando el comportamiento de la aplicación varía rápidamente dentro de la ventana, precisamente el escenario de interés en aplicaciones multifásicas [8]. Por ello, mantener el conjunto de eventos dentro del presupuesto físico de la plataforma no es solo una decisión de cobertura, sino de calidad de la medida.

Eventos genéricos y eventos crudos

Los eventos genéricos son una abstracción portable que el kernel traduce a la codificación concreta de la microarquitectura; los eventos crudos se especifican mediante la codificación nativa del procesador (selector de evento, máscara de sub-evento, calificadores adicionales). Cuando el kernel carece de una entrada de traducción para un modelo de procesador reciente, la vía genérica falla aunque el contador físico exista, y acceder a él exige codificación cruda — trasladando al instrumento la responsabilidad de garantizar que esa codificación es correcta para esa microarquitectura específica. Una codificación incompleta puede abrir el contador sin error aparente y producir valores sin sentido físico, un modo de fallo más peligroso que un error explícito porque no se manifiesta hasta examinar la coherencia de los datos.

Contadores de núcleo y contadores de *uncore*

Los contadores de núcleo observan eventos atribuibles a un núcleo concreto y pueden asociarse a un proceso. Los contadores de *uncore* observan recursos compartidos del zócalo — controlador de memoria, caché de último nivel distribuida, interconexiones — y por tanto son

de alcance de sistema, no atribuibles a un proceso específico. Los contadores del controlador de memoria observan el tráfico que efectivamente alcanza memoria principal, con independencia de si lo originó un acceso de demanda o precarga, por lo que constituyen la fuente empleada en este trabajo para el denominador de la ecuación (1.4) (sección 1.1.2) — aunque su acceso está sujeto a una categoría de privilegio distinta (sección 1.1.5).

Métricas microarquitectónicas de eficiencia

Instrucciones por ciclo (IPC), tasa de fallos de caché y ciclos de estancamiento (*stall cycles*) son señales complementarias que describen cómo se distribuye el costo de ejecución entre cómputo y memoria [9, 10]. El IPC mide cuánto ciclo de reloj se traduce en trabajo útil retirado [11, Ch. 6]; la tasa de fallos de último nivel describe la presión relativa sobre la jerarquía de memoria [11, Ch. 2]; y los *stall cycles* asociados a la etapa de ejecución — espera por operandos desde memoria, a diferencia de los originados en captación/decodificación — son la señal con mayor valor discriminante para el régimen *memory-bound* [11, Ch. 6]. Ninguna constituye por sí sola una formulación analítica exacta de los regímenes, pero en conjunto describen patrones de ejecución consistentes con cada uno [10].

Ventanas de muestreo y naturaleza temporal de la observación

Los contadores de hardware son acumulativos: crecen monótonamente desde que se habilitan. La unidad de análisis adecuada no es una lectura individual, sino la diferencia entre dos lecturas consecutivas — una ventana de muestreo —, que describe el trabajo realizado en ese intervalo. La elección del período de muestreo es un compromiso: un período corto ofrece mayor resolución temporal pero reduce el número de eventos acumulados por ventana, degradando la relación señal–ruido de las métricas derivadas; un período largo produce métricas más estables pero promedia sobre intervalos que pueden abarcar varias fases distintas. Adicionalmente, el intervalo efectivo entre lecturas no coincide exactamente con el período nominal solicitado — el hilo de muestreo está sujeto a planificación de un sistema operativo de propósito general —, por lo que toda métrica derivada debe calcularse sobre el intervalo realmente transcurrido, medido con un reloj monótono.

1.1.4. Espacios de ejecución: *user-space* y *kernel-space*

Los sistemas operativos modernos separan la ejecución en dominios de privilegio diferenciados, apoyados en modos de operación del procesador: en modo usuario, el software solo accede a memoria de espacio de usuario; en modo kernel, puede acceder también al espacio del kernel [12, ch. 2]. Operaciones sensibles — gestión de memoria, acceso a hardware, entra-

da/salida — requieren privilegios de kernel, lo que impide que procesos ordinarios interfieran con estructuras críticas [12, ch. 2]. Para el software de aplicación esto implica que el acceso a recursos privilegiados ocurre mediante mecanismos de mediación — en sistemas Unix/Linux, principalmente llamadas al sistema [13] —, así que cualquier mecanismo de monitoreo o control implementado fuera del kernel debe apoyarse en interfaces explícitamente expuestas por la plataforma.

1.1.5. Interfaces estándar de observabilidad e instrumentación

La observabilidad de sistemas heterogéneos CPU–GPU depende de interfaces que expongan, desde *user-space*, métricas de rendimiento, utilización y consumo sin modificar el kernel ni los controladores. Las plataformas modernas exponen `perf_event` para contadores de CPU, Intel RAPL para energía de CPU, y NVIDIA Management Library (NVML) para telemetría de GPU [4, 14, 15, 16].

La interfaz `perf_event` y los modos de atribución

`perf_event` es la infraestructura estándar en Linux para acceder a la PMU desde *user-space* [14]. El alcance de atribución de la medida es determinante para la validez experimental: la medición de alcance de CPU contabiliza todo lo que ocurre en un procesador dado, incluida actividad ajena al experimento; la medición por proceso contabiliza únicamente ese proceso, con independencia de sobre qué CPU se planifique. Cuando la carga es paralela, la interfaz permite heredar el contador a los descendientes creados tras habilitarlo, de modo que la medida abarque el árbol completo de ejecución y solo ese árbol — combinación de atribución por proceso con herencia, adoptada en este trabajo. Una restricción asociada: con herencia habilitada la interfaz no admite lectura agrupada de varios contadores en una operación, así que cada evento se lee de forma independiente, introduciendo un desfase de orden microsegundo entre lecturas de una misma ventana — despreciable frente a períodos de muestreo del orden del milisegundo, pero declarado como característica conocida del instrumento.

Niveles de privilegio en el acceso a contadores

El acceso a la infraestructura de monitorización desde espacio de usuario no es irrestricto: Linux expone un parámetro de configuración que restringe progresivamente las capacidades disponibles para un usuario sin privilegios especiales, y en sus niveles más restrictivos permite medir contadores del propio proceso pero no eventos de alcance de sistema — categoría de los contadores de *uncore* (sección 1.1.3). Esto tiene una consecuencia metodológica directa: la disponibilidad de la medición depende no solo del hardware, sino de la política de admi-

nistración del sistema, así que caracterizar las capacidades efectivas de la plataforma es un paso previo obligatorio al diseño experimental.

Intel RAPL

RAPL (*Running Average Power Limit*) expone energía acumulada para dominios como paquete del procesador, núcleos y DRAM [15, 17]. Reporta mediante registros específicos del modelo, en unidades dependientes de la plataforma, sin cobertura exhaustiva del consumo total del nodo ni disponibilidad uniforme entre microarquitecturas [17]. Al ser acumulativos, los registros pueden desbordarse, lo que exige que el instrumento registre el valor máximo del rango antes de capturar, para distinguir un desbordamiento real de un consumo negativo espurio. RAPL es una capacidad del procesador, no una interfaz universal: microarquitecturas anteriores a su introducción carecen de ella sin alternativa por software, por lo que su disponibilidad es criterio de admisibilidad para cualquier plataforma de este tipo de estudio.

NVIDIA Management Library

NVML permite consultar el estado operativo de una GPU NVIDIA — utilización, potencia, memoria y otros parámetros del dispositivo [16] —, orientada a la telemetría del acelerador como dispositivo completo y no a eventos microarquitectónicos de un kernel concreto [4, 16]. Esta diferencia de granularidad es relevante para el problema abordado: la utilización que NVML reporta indica qué fracción del tiempo hubo trabajo activo, no cuántas operaciones por byte realizó ese trabajo, así que caracterizar el régimen de ejecución en GPU exige una estrategia distinta a la de CPU (sección 2.3.4).

1.1.6. Gestión dinámica de frecuencia en sistemas operativos: DVFS, P-states y *governors*

DVFS ajusta el punto operativo del hardware mediante cambios coordinados de frecuencia y voltaje, apoyado en reguladores de voltaje y unidades de control de potencia dedicadas que ejecutan las transiciones [18]. El estándar ACPI formaliza esto mediante *performance states* (P-states, niveles operativos dentro del estado activo, con P0 como el de mayor rendimiento) y *C-states* (niveles de inactividad) [18, 11].

Sobre esta base operan los *governors*: políticas que deciden cuándo y cómo transicionar entre P-states, sin implementar el mecanismo físico. Las políticas de frecuencia fija (*performance*, *powersave*) sitúan al procesador en un extremo operativo; los gobernadores dinámicos (*ondemand*, *conservative*, *schedutil*) ajustan la frecuencia según carga observa-

da o, en el caso de `schedutil`, integrada con el planificador [18]. En Linux, dos controladores dominan: `acpi-cpufreq`, genérico, que expone un gobernador `userspace` capaz de fijar una frecuencia objetivo explícita; e `intel_pstate`, por defecto en muchas generaciones Intel, que solo soporta `performance/powersave` — sin gobernador `userspace` —, de modo que fijar un punto operativo concreto se logra restringiendo el rango permitido mediante límites inferior y superior, no escribiendo una frecuencia objetivo directamente [18]. Esta diferencia tiene consecuencias operativas directas: un agente de control portable debe detectar el controlador activo y adaptar su estrategia de actuación en consecuencia. Adicionalmente, las transiciones entre P-states no son instantáneas — del orden de 10 ms, o 1 ms con *hardware P-states* [18] —, por lo que una política de control debe amortizar ese costo y no decidir a un ritmo superior al que el sistema puede materializar.

Dominios de frecuencia y aislamiento experimental

El dominio de frecuencia — el conjunto de procesadores lógicos afectados simultáneamente por una misma solicitud de cambio — puede abarcar un zócalo completo o reducirse a cada núcleo individual, según la generación de hardware. Si el dominio excede los núcleos asignados al experimento, la actuación puede alterar procesos ajenos y viceversa, comprometiendo tanto la validez interna del experimento como la convivencia en infraestructura compartida. Por ello, el dominio real de frecuencia — consultado en la plataforma, no supuesto a partir del modelo de procesador — es una verificación previa obligatoria antes de habilitar cualquier actuación. De forma relacionada, cuando *simultaneous multithreading* comparte los recursos de un núcleo físico entre dos procesadores lógicos, la política de asignación debe declararse explícitamente y verificarse contra la topología real, para evitar que hilos de medición y de carga compitan por los mismos recursos.

Particularidades de DVFS en GPU

En GPU, DVFS no se reduce al esquema de CPU: el comportamiento energético y temporal depende de la interacción entre al menos dos dominios funcionales — procesamiento (multiprocesadores de *streaming*) y memoria del dispositivo —, que no responden igual a un cambio de frecuencia [19]. Cuando el rendimiento está limitado por cómputo, la frecuencia del dominio de procesamiento impacta más directamente el tiempo de ejecución; cuando predomina la limitación por memoria, esa sensibilidad disminuye y el dominio de memoria pasa a ser más determinante [19, 5]. Además, potencia, rendimiento y energía no varían de forma independiente con la frecuencia: bajar la frecuencia puede reducir la potencia instantánea sin reducir la energía total si el tiempo de ejecución aumenta demasiado, y viceversa [19, 5].

1.1.7. Cargas de trabajo: kernels, benchmarks y microbenchmarks

En HPC, un kernel es una rutina computacional bien delimitada que concentra una fracción relevante del trabajo de una aplicación — una multiplicación de matrices, una transformada de Fourier, una actualización de *stencil* [20]. Al encapsular un patrón dominante de ejecución más estable que la aplicación completa, el kernel es una unidad de observación especialmente adecuada para analizar regímenes *compute-bound* y *memory-bound*.

Un benchmark es una carga diseñada o seleccionada para medir y comparar el comportamiento de un sistema bajo condiciones reproducibles, facilitando comparación entre arquitecturas o políticas de control [11, Ch. 12]. Un microbenchmark es una forma más reducida, orientada a aislar el comportamiento de un componente o mecanismo específico — por ejemplo, ancho de banda de memoria o densidad aritmética [11, Ch. 12] —, y es el instrumento adecuado para la calibración Roofline de la sección 1.1.2, mientras que los benchmarks representativos constituyen el objeto de caracterización.

Las suites de benchmarks científicos suelen reportar, al finalizar, una medida agregada de trabajo realizado — pero esa medida no siempre corresponde a operaciones de punto flotante: algunos programas cuantifican su trabajo en unidades distintas (números aleatorios, claves ordenadas) según la naturaleza del problema. Como el numerador de la intensidad operacional (ecuación (1.4)) debe expresarse en FLOPs para ser comparable con un techo calibrado en esas unidades, verificar explícitamente la unidad reportada por cada carga es un criterio de admisión al catálogo experimental, no un detalle de implementación (sección 2.2.1).

1.1.8. Aprendizaje automático supervisado ligero

El aprendizaje supervisado infiere una función que mapea variables de entrada hacia una salida a partir de datos etiquetados, dividiendo el conjunto en entrenamiento y prueba para evaluar generalización [21]. En contextos con restricciones de tiempo y recursos, un clasificador ligero es aquel cuya inferencia exige baja complejidad computacional y memoria acotada, sin sacrificar capacidad razonable de generalización. Entre los clasificadores supervisados clásicos más relevantes para este problema se encuentran:

Bosques Aleatorios (*Random Forest*). Método de aprendizaje en conjunto que combina múltiples árboles de decisión entrenados de forma independiente, agregando sus predicciones (votación mayoritaria en clasificación) [22]. Dos mecanismos estocásticos — *bootstrap* sobre las muestras de entrenamiento y selección aleatoria de un subconjunto de variables en cada

división — garantizan baja correlación entre árboles, mitigando el sobreajuste característico de árboles individuales profundos [22].

Regresión Logística. Estima la probabilidad de pertenencia a una clase mediante un predictor lineal transformado por la función sigmoide:

$$z = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n, \quad P(y = 1 \mid x) = \frac{1}{1 + e^{-z}} \quad (1.5)$$

Sus ventajas son simplicidad, bajo costo de inferencia e interpretabilidad; su limitación principal es la frontera de decisión lineal, insuficiente cuando la separación entre clases depende de relaciones fuertemente no lineales [23].

XGBoost. Ensamble de árboles potenciados de forma secuencial y aditiva, donde cada nuevo árbol corrige los residuos del modelo actual, incorporando regularización directamente en el objetivo de aprendizaje para controlar la complejidad [24]. Frente a la regresión logística es más expresivo; frente a un árbol individual, más robusto — a costa de mayor complejidad estructural.

1.1.9. Producto Energía–Retardo (EDP)

Evaluar una estrategia de optimización energética exige una métrica que integre energía y rendimiento simultáneamente: minimizar solo una de las dos puede resultar globalmente inconveniente (ahorro energético con degradación severa, o máximo desempeño con costo energético desproporcionado). El Producto Energía–Retardo cumple ese rol [25]:

$$EDP = E \cdot T^w \quad (1.6)$$

donde E es la energía consumida, T el tiempo de ejecución y w un factor de ponderación; $w = 1$ da el EDP clásico, $w = 2$ una variante que penaliza más fuertemente la degradación temporal (*Energy–Delay² Product*) [5, 25]. Su utilidad radica en evitar decisiones sesgadas por una sola dimensión: reducir únicamente la potencia puede inducir pérdidas de rendimiento que terminen aumentando la energía total consumida, por lo que una función multiobjetivo basada en EDP resulta más adecuada para seleccionar configuraciones DVFS [5].

1.2. Marco Legal

1.3. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. En este contexto, el DVFS y las técnicas afines de *frequency capping* y *power capping* se han consolidado como mecanismos de control relevantes tanto en procesadores como en aceleradores GPU. Sin embargo, la eficacia de estos mecanismos depende de forma crítica del comportamiento dinámico de la carga de trabajo, lo que ha impulsado el desarrollo de modelos de caracterización, predicción y control cada vez más sofisticados [1, 2, 4, 5, 19].

Una primera línea de trabajo corresponde a la evaluación empírica del impacto de DVFS sobre aplicaciones y kernels HPC. Calore et al. analizaron técnicas DVFS sobre procesadores y aceleradores modernos desde el punto de vista del usuario, mostrando que el beneficio energético no es uniforme y que depende del carácter *compute-bound* o *memory-bound* de las partes críticas de la aplicación. Este resultado es relevante porque confirma que la optimización energética no puede formularse como una política global única, sino como una decisión sensible al perfil de ejecución de cada kernel o fase [4]. En una línea similar, Simsek et al. estudiaron simulaciones astrofísicas sobre GPU y mostraron que la instrumentación del código y el ajuste estático y dinámico de frecuencia permiten reducir el consumo energético con pérdidas moderadas de rendimiento, reportando reducciones de energía por GPU de hasta 7.82 % con una pérdida de rendimiento de 2.95 % [1]. Más recientemente, Costa et al. evaluaron *frequency capping* y *power capping* en un sistema exascale, observando que la efectividad relativa de cada mecanismo depende tanto de la naturaleza de la aplicación como de la escala de ejecución; en particular, *frequency capping* tendió a ofrecer mejores resultados energéticos a escala, mientras *power capping* resultó más adecuado en ciertos escenarios de nodo único o utilización irregular [2].

Una segunda línea de investigación se centra en los métodos *offline* de selección de frecuencia óptima. En este grupo destaca el trabajo de Guerreiro et al., quienes propusieron un esquema de clasificación de aplicaciones GPGPU consciente de DVFS, capaz de inferir, a partir de eventos de hardware recolectados a una sola frecuencia, cómo cambiarán tiempo de ejecución, potencia y energía al recorrer el resto del espacio de frecuencias. Su propuesta permitió encontrar pares de frecuencias casi óptimos, con ahorros medios de energía de 16 % a 20 % y picos de hasta 36 %, según la arquitectura evaluada [19]. De forma complementaria,

Ali et al. desarrollaron un método automatizado y portable para seleccionar la frecuencia óptima de GPU a partir de tres etapas: caracterización de características, modelado analítico de potencia y tiempo de ejecución, y selección multiobjetivo mediante EDP y ED²P. Su metodología requiere recolectar métricas en una configuración base y luego estimar el comportamiento en el resto del espacio DVFS, alcanzando ahorros promedio de 29.6% de energía con 5.2% de pérdida de rendimiento en una arquitectura y 22.6% con 4.7% en otra [5]. Estos trabajos constituyen antecedentes sólidos porque muestran que el espacio DVFS puede explorarse sin fuerza bruta y con buena portabilidad. No obstante, comparten una limitación importante: su lógica de decisión es esencialmente *offline* o precomputada, por lo que no aborda de manera explícita la adaptación fina a la multifasicidad intra-ejecución de aplicaciones HPC generales.

Una tercera línea aborda la caracterización y clasificación de cargas o kernels a partir de telemetría de bajo nivel. Shekofteh et al. demostraron que la clasificación de kernels GPU puede realizarse con un conjunto reducido de métricas, seleccionadas para minimizar *overhead* sin sacrificar precisión, lo que confirma que la identificación *online* de comportamientos como *compute-bound* y *memory-bound* es técnicamente viable. Sin embargo, su objetivo principal fue mejorar la planificación y co-ejecución de kernels, no gobernar DVFS [9]. De manera análoga, Littman y Deakin exploraron el uso de aprendizaje automático para clasificar límites de rendimiento a partir de contadores de rendimiento, apoyando la idea de que la frontera *compute-bound/bandwidth-bound* puede inferirse desde datos y no solo mediante inspección manual del modelo Roofline [10]. En conjunto, estos trabajos respaldan la hipótesis de que una fase de ejecución puede representarse mediante una firma microarquitectónica observable, aunque no proponen por sí mismos un lazo de control energético que traduzca dicha clasificación en decisiones DVFS.

La cuarta línea corresponde a los mecanismos *online* de control dinámico durante ejecución. El antecedente más fuerte en esta categoría es DRLCAP, que propone un marco general de *frequency capping* de GPU en tiempo de ejecución basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema para detectar cambios de fase, aprende una política adaptativa y reduce en promedio 22% de la energía de GPU con menos de 3% de degradación en arquitecturas NVIDIA, además de reportar resultados sobre AMD [26]. Este trabajo demuestra que el control *online* de frecuencia de GPU es factible y efectivo, pero también deja clara una diferencia conceptual frente a enfoques más ligeros: se trata de una política basada en aprendizaje por refuerzo profundo, orientada a GPU, sin formular explícitamente el problema como clasificación supervisada ligera de fases seguida de una política discreta interpretable. Por tanto, aunque constituye un antecedente directo, no

agota el espacio de soluciones posibles para agentes ligeros en espacio de usuario.

Finalmente, una restricción frecuentemente subestimada en la literatura es el *overhead* del propio mecanismo de control. Velicka et al. mostraron que la latencia de conmutación de frecuencia en GPU no es despreciable, varía entre arquitecturas y entre pares de frecuencias, y puede llegar a ser suficientemente alta como para comprometer el beneficio de políticas excesivamente reactivas [27]. Esta observación es central, porque implica que un esquema *online* no debe limitarse a detectar fases correctamente: también debe amortizar el costo del cambio de frecuencia y evitar decisiones demasiado finas o frecuentes.

Aun cuando la literatura reciente ha avanzado en control dinámico de frecuencia durante ejecución, la mayoría de los trabajos revisados se concentra en el dominio GPU de forma aislada. DRLCAP, por ejemplo, propone un marco *online* guiado por aprendizaje por refuerzo profundo, pero su problema de control está centrado en la GPU y no en una política coordinada sobre CPU y GPU dentro de un nodo heterogéneo [26]. Del mismo modo, los métodos de selección óptima de frecuencia de Ali et al. y los modelos de clasificación conscientes de DVFS de Guerreiro et al. se enfocan en la selección de configuraciones para GPU [5, 19]. Aunque existen propuestas de coordinación entre CPU, GPU y memoria, como SparseDVFS, estas se desarrollan en el contexto de inferencia de redes neuronales en dispositivos *edge*, con una lógica de partición por bloques y una señal de control centrada en la esparsidad de operadores, por lo que su alcance, supuestos de carga y entorno de despliegue difieren sustancialmente del problema de HPC científico general abordado aquí [28].

En conjunto, esta revisión muestra que la literatura ya dispone de componentes importantes del problema, aunque todavía de forma fragmentada: existen métodos *offline* robustos para seleccionar frecuencias óptimas, mecanismos de caracterización y clasificación de cargas, y controladores *online* avanzados para GPU. Sin embargo, sigue abierta una brecha en torno a soluciones ligeras, implementables en espacio de usuario, que integren caracterización *online* de fases de ejecución, telemetría estándar de baja intrusión y decisiones DVFS coordinadas sobre CPU y GPU en nodos heterogéneos de alto rendimiento. En ese sentido, el aporte de este trabajo no radica en afirmar que la clasificación de fases o el control *online* sean completamente inéditos, sino en proponer una integración distinta: un agente ligero, implementable sin modificaciones al kernel, que utilice modelos supervisados clásicos para inferir el régimen de ejecución y traduzca esa inferencia en decisiones DVFS de bajo *overhead*, con foco explícito en la optimización del Producto Energía–Retardo.

Capítulo 2

Metodología

2.1. Enfoque metodológico

La presente investigación se desarrolla bajo un enfoque cuantitativo de tipo experimental aplicado, orientado al cumplimiento del objetivo general del proyecto. El propósito metodológico consiste en determinar en qué medida un agente de gestión DVFS permite maximizar la eficiencia energética sin degradar significativamente el rendimiento de la aplicación en ejecución. Dicho impacto se evalúa a través de métricas cuantificables y reproducibles, principalmente el tiempo de ejecución, la energía consumida y el Producto Energía–Retardo (EDP).

El carácter experimental se sustenta en la manipulación deliberada de una variable independiente — el estado de frecuencia impuesto al procesador — sobre un conjunto controlado de cargas de trabajo, observando su efecto sobre variables dependientes de rendimiento y consumo. El carácter aplicado responde a que el producto de la investigación no es únicamente conocimiento descriptivo sobre el comportamiento energético de las cargas, sino un artefacto software funcional cuya utilidad se evalúa empíricamente frente a las alternativas ya disponibles en el sistema operativo.

El desarrollo se estructura en cuatro fases secuenciales e interdependientes, que abarcan desde la caracterización de cargas y extracción de telemetría hasta la validación empírica del sistema. Este capítulo describe en detalle el diseño de la primera fase, por ser la que se encuentra ejecutada al momento de redacción del presente documento, y presenta el diseño previsto para las fases restantes.

2.2. Diseño metodológico

2.2.1. Población y muestra

La población de estudio está constituida por el conjunto de aplicaciones ejecutables en sistemas heterogéneos CPU–GPU. Debido a la imposibilidad de abarcar la totalidad de estas, se define una muestra no probabilística de tipo intencional, compuesta por benchmarks y microbenchmarks representativos de regímenes de ejecución contrastantes. Esta selección permite inducir comportamientos diferenciados y obtener evidencia experimental suficiente para construir un conjunto de datos útil para el entrenamiento del clasificador. El uso de benchmarks representativos y de una muestra intencional es coherente con la necesidad de trabajar con cargas controladas y reproducibles dentro del alcance de un proyecto de pregrado.

El criterio de inclusión en el catálogo experimental combina tres condiciones. En primer lugar, la carga debe disponer de una verificación interna de corrección que permita descartar ejecuciones inválidas, de modo que una corrida cuyo resultado numérico no sea correcto no contamine el conjunto de datos. En segundo lugar, la carga debe reportar de forma explícita el volumen de trabajo realizado, requisito necesario para estimar el numerador de la intensidad operacional según lo descrito en la sección 1.1.2. En tercer lugar, ese volumen de trabajo debe estar expresado en operaciones de punto flotante y no en otra unidad (números aleatorios generados, claves ordenadas), pues de lo contrario no sería comparable con un techo Roofline calibrado en FLOPs; las cargas que cuantifican su trabajo en otras unidades quedan excluidas del catálogo de caracterización, con independencia de su relevancia como benchmarks en otros contextos.

2.2.2. Plataforma experimental

Los experimentos se ejecutan sobre un nodo de cómputo heterogéneo cuyas características se resumen en la Tabla 2.1. La selección de la plataforma no fue arbitraria: además de la disponibilidad de un acelerador gráfico instrumentable mediante NVML, se estableció como condición de admisibilidad la presencia de una interfaz de medición energética interna accesible sin privilegios administrativos, dado que, como se argumentó en el marco conceptual, dicha capacidad es una propiedad del procesador que no admite sustitución por software.

La identidad precisa de la microarquitectura no es un dato meramente descriptivo en este trabajo: varios de los eventos crudos de PMU que el instrumento utiliza (sección 1.1.5) carecen de mapeo genérico en el kernel de este nodo y se abren mediante una codificación específica

del procesador, verificada empíricamente y restringida en el código a coincidir exactamente con el par `cpu family/model` de la Tabla 2.1; ese mismo gateo es lo que impide, por diseño, que el instrumento aplique sin verificación una codificación cruda a un nodo distinto de este.

Tabla 2.1: Características de la plataforma experimental.

Componente	Descripción
Procesador	$2 \times$ Intel Xeon Gold 5315Y
Microarquitectura	Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)
Identificador CPUID (<code>cpu family/model</code>)	6 / 106 — el mismo par leído en tiempo de ejecución desde /
Extensiones SIMD relevantes	AVX2, FMA3, AVX-512 (<code>avx512f</code> , <code>avx512bw</code> , <code>avx512cd</code> , <code>avx512er</code>)
Núcleos	8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)
Nodos NUMA	2 (uno por zócalo)
Jerarquía de caché	L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB
Tamaño de línea de caché	64 bytes
Controlador de frecuencia	<code>intel_pstate</code>
Rango de frecuencia	0.8–3.6 GHz
Dominio de frecuencia	Por núcleo lógico
Medición energética	Intel RAPL (dominios de paquete y DRAM por zócalo)
Acelerador	NVIDIA A100 PCIe 40 GB
Gestor de recursos	Slurm

Las condiciones de ejecución se controlan mediante asignación exclusiva del nodo a través del gestor de recursos, de modo que ninguna carga ajena al experimento comparta el procesador durante las mediciones. Los procesadores lógicos empleados se restringen a hilos primarios de un mismo zócalo, evitando la asignación simultánea de hilos hermanos del mismo núcleo físico, de acuerdo con lo expuesto en la sección 1.1.6.

2.2.3. Instrumentos de recolección

La recolección de telemetría se implementa mediante un instrumento propio, desarrollado específicamente para este trabajo, compuesto por dos capas con responsabilidades separadas.

La capa inferior es un ejecutor escrito en C++ que lanza la carga de trabajo como proceso hijo y adjunta los contadores de hardware sobre ella. El diseño de esta capa responde directamente a los requisitos de atribución expuestos en la sección 1.1.5: el proceso hijo se detiene inmediatamente después de su creación y antes de ejecutar la primera instrucción de

la carga, momento en el cual el proceso padre abre los contadores asociados a ese identificador de proceso con herencia habilitada; solo entonces se reanuda la ejecución. Este protocolo garantiza que ninguna instrucción de la carga medida se ejecute fuera del intervalo de observación y que la medida abarque el árbol completo de procesos e hilos que la carga genere, sin incluir actividad ajena.

El muestreo se realiza mediante un hilo productor dedicado, fijado a un procesador lógico distinto de los asignados a la carga, que a intervalos regulares lee los contadores y deposita las muestras en una estructura circular de productor y consumidor únicos. Un hilo consumidor, igualmente fijado a un procesador lógico separado, extrae las muestras y las persiste. Esta separación evita que la actividad de escritura a disco interfiera con la periodicidad del muestreo, y el aislamiento de ambos hilos respecto de los núcleos medidos reduce su interferencia sobre la carga observada. La ausencia de pérdidas en la estructura circular se registra como indicador de integridad de cada corrida.

La capa superior es un orquestador escrito en Python, responsable de la lógica experimental: interpreta el manifiesto que define la campaña, detecta las capacidades de la plataforma, ejecuta las verificaciones previas, realiza la calibración, planifica y lanza cada combinación experimental, aplica el estado de frecuencia correspondiente, valida el resultado de cada corrida y transforma las muestras crudas en el conjunto de datos final. La separación entre ambas capas responde a un criterio de responsabilidad: la capa C++ resuelve la medición de baja latencia, donde el costo de la instrumentación es crítico, mientras que la capa Python resuelve la lógica experimental, donde prima la claridad y la trazabilidad sobre el rendimiento.

2.2.4. Variables observadas

Antes de describir las señales capturadas, conviene precisar qué se entiende por ventana de muestreo, término empleado de aquí en adelante. El hilo productor descrito en la sección anterior toma una lectura de los contadores cada cierto período configurado en la campaña; cada lectura individual es un valor acumulativo, y por sí sola no describe el comportamiento de un intervalo concreto. Una ventana de muestreo se define como el intervalo delimitado por dos lecturas consecutivas, y su contenido — el trabajo realizado durante ese intervalo — se obtiene restando la lectura anterior de la lectura actual, conforme a lo argumentado en la sección 1.1.3. Por tanto, una ventana no es un conjunto o agregado de varias muestras, sino el resultado de un único par de lecturas consecutivas; se requieren dos lecturas para producir una ventana, de modo que la primera lectura de cada corrida no genera ventana alguna, al carecer de una lectura previa contra la cual diferenciarse (condición registrada en la Tabla 2.5 como “sin diferencia previa”). La duración real de una ventana corresponde al

tiempo efectivamente transcurrido entre ambas lecturas, medido con un reloj monótono, y no al período nominal configurado.

La Tabla 2.2 resume las señales de telemetría capturadas en cada ventana de muestreo. El conjunto de eventos se dimensionó para permanecer dentro del presupuesto de contadores físicos simultáneos de la plataforma, verificado empíricamente sobre la máquina en lugar de asumirse a partir del modelo de procesador, con el fin de evitar el error de extrapolación asociado a la multiplexación descrito en la sección 1.1.3.

Tabla 2.2: Señales de telemetría capturadas por ventana de muestreo.

Señal	Propósito
Instrucciones retiradas	Base para IPC y tasa de instrucciones.
Ciclos de reloj	Base para IPC y para la fracción de ciclos de estancamiento.
Referencias a caché	Denominador de la tasa de fallos.
Fallos de caché	Estimación de bytes movidos y de la presión sobre memoria.
Ciclos de estancamiento en ejecución	Señal directa de espera por operandos, asociada al régimen <i>memory-bound</i> .
Operaciones de punto flotante retiradas (escalar, 128 bits, 256 bits, 512 bits)	Medición directa de FLOPs por ventana, ponderada por ancho de registro vectorial (ecuación (2.1)); fuente principal del numerador de la intensidad operacional.
Tiempo habilitado y tiempo contando	Diagnóstico de multiplexación y escalado de la medida.
Energía acumulada por dominio RAPL	Cálculo de energía y potencia por ventana.
Frecuencia efectiva del núcleo representativo	Lectura instantánea de <code>scaling_cur_freq</code> en el mismo ciclo de muestreo de los contadores de PMU; permite contrastar durante la carga el nivel solicitado contra el realmente observado.

A partir de estas señales se derivan, para cada ventana, las métricas de eficiencia descritas en el marco conceptual — instrucciones por ciclo, tasa de fallos del último nivel de caché, fallos por cada mil instrucciones, fracción de ciclos de estancamiento — así como la intensidad operacional y las magnitudes energéticas. Todas las tasas se calculan sobre el intervalo

realmente transcurrido entre lecturas consecutivas, conforme a lo argumentado en la sección 1.1.3.

2.3. Fase 1: Recolección de información y caracterización de cargas

La primera fase tiene por objetivo construir el conjunto de datos etiquetado que sustentará el entrenamiento del clasificador, junto con el instrumento y el protocolo que garantizan su validez. Se describe a continuación el procedimiento completo.

2.3.1. Verificación previa de la plataforma

Antes de ejecutar cualquier medición se aplica una batería de verificaciones automáticas sobre el nodo, orientada a confirmar que las condiciones supuestas por el diseño experimental se cumplen efectivamente. Estas verificaciones se agrupan en cinco familias: capacidades del entorno (estado de las tecnologías de aceleración de frecuencia, afinidad NUMA de los núcleos asignados, política de multihilo declarada, dominio real de frecuencia, número de contadores simultáneos disponibles); integridad del catálogo (existencia y permisos de ejecución de cada binario, correspondencia de su suma de verificación, validez del criterio de éxito declarado, suficiencia de memoria); disponibilidad de instrumentación (dominios energéticos solicitados frente a los presentes, corrección del manejo de desbordamiento); condiciones de almacenamiento y presupuesto; y, cuando la campaña contempla control de frecuencia, permisos efectivos de escritura y cobertura del dominio.

Las verificaciones se clasifican en bloqueantes y no bloqueantes. Una verificación bloqueante que no se satisface detiene la campaña antes de tocar el sistema, bajo el principio de que es preferible no producir datos a producir datos cuya validez no puede sostenerse. Este mecanismo se ejecuta de forma automática como parte del comando que lanza la campaña, y no como un paso manual previo, de modo que no exista la posibilidad de iniciar una medición sin haberlo aplicado.

2.3.2. Catálogo de cargas de trabajo

El catálogo experimental se compone de cargas de dos roles distintos. Las cargas de calibración sirven para determinar empíricamente los techos del modelo Roofline sobre la plataforma: un microbenchmark de ancho de banda de memoria, cuyo conjunto de trabajo excede ampliamente la capacidad de la caché de último nivel para garantizar que el tráfico

alcance la memoria principal, proporciona la estimación de BW_{pico} ; y un microbenchmark de densidad aritmética sobre datos residentes en caché proporciona la estimación de P_{pico} . Las cargas de conjunto de datos son los benchmarks cuyo comportamiento se pretende caracterizar.

La Tabla 2.3 resume la composición del catálogo. Todas las cargas de conjunto de datos provienen de suites de benchmarks científicos de amplia adopción, se emplean sin modificar su código fuente y se compilan sobre la plataforma experimental con las opciones por defecto de cada suite, registrando la suma de verificación del binario resultante. Este registro cumple una función metodológica precisa: permite confirmar, antes de cada corrida individual, que el binario ejecutado es exactamente el mismo que se caracterizó, descartando la posibilidad de que una recompilación silenciosa altere las condiciones a mitad de campaña. Dado que un mismo código fuente compilado con cadenas de herramientas distintas produce binarios funcionalmente equivalentes pero con sumas de verificación diferentes, este registro se mantiene asociado a la plataforma en la que se realizó la compilación.

Tabla 2.3: Composición del catálogo experimental de cargas de trabajo.

Rol	Cantidad	Propósito
Calibración	2	Estimación empírica de BW_{pico} (ancho de banda sostenido de memoria) y de P_{pico} (densidad aritmética sobre datos en caché).
Conjunto de datos	6	Kernels de una suite de benchmarks científicos paralelos, seleccionados por reportar su trabajo en operaciones de punto flotante y cubrir distintos patrones de acceso a memoria (resolución de sistemas por métodos implícitos, multigrid, gradiente conjugado, transformada rápida de Fourier).
Conjunto de datos	1	Multiplicación densa de matrices sobre una biblioteca de álgebra lineal optimizada, como referencia de régimen intensivo en cómputo.

2.3.3. Calibración y criterio de etiquetado

La calibración se ejecuta al inicio de cada campaña, sobre la misma asignación de núcleos y con la misma política de afinidad que se emplea después en las corridas de caracterización, de

modo que los techos obtenidos correspondan a la configuración realmente utilizada. A partir de los valores medidos se calcula el punto de inflexión según la ecuación (1.3). Como control de validez, los valores obtenidos se contrastan contra una referencia empírica declarada en el manifiesto de la campaña, y una discrepancia superior a una tolerancia establecida detiene la ejecución: esta verificación protege frente a calibraciones realizadas bajo condiciones anómalas que pasarían inadvertidas si el valor resultante se aceptara sin contraste.

Sobre esta base se define el etiquetado. Cada ventana de muestreo recibe una etiqueta derivada exclusivamente de la comparación entre su intensidad operacional observada y el punto de inflexión calibrado, sin intervención de juicio manual. Adicionalmente, cada carga del catálogo declara una expectativa cualitativa de régimen, proveniente de la literatura sobre la suite correspondiente. Es importante subrayar que ambas anotaciones cumplen funciones distintas y no deben confundirse: la etiqueta derivada del modelo constituye la variable objetivo del problema de clasificación, mientras que la expectativa declarada actúa únicamente como control de coherencia. La correspondencia entre ambas, evaluada de forma agregada por carga, permite detectar errores sistemáticos del instrumento, pero la expectativa nunca sustituye ni corrige la etiqueta medida.

Como control adicional de estabilidad, cada campaña incluye repeticiones de una carga de referencia, sobre las cuales se calcula la dispersión relativa de las métricas de eficiencia. Una dispersión elevada indica que las condiciones de medición no son suficientemente estables como para sostener comparaciones finas entre corridas, y se registra como advertencia asociada a la campaña.

2.3.4. Caracterización de intensidad operacional y calibración Roofline en GPU

A diferencia de CPU, en GPU **no existe una medición de FLOPs por ventana**. La telemetría muestreada durante la corrida en el dominio GPU proviene exclusivamente de NVIDIA Management Library (NVML), y por cada muestra produce únicamente potencia, utilización (de cómputo y de memoria), reloj del multiprocesador de streaming, temperatura y energía acumulada del dispositivo — ninguna de estas señales cuantifica cuántas operaciones de punto flotante realizó el kernel ni cuántos bytes movió. La intensidad operacional de cada carga GPU se obtiene, en cambio, mediante un procedimiento distinto y separado, más cercano en espíritu a la calibración de los techos del modelo Roofline que a un muestreo continuo: una caracterización dinámica *fuera de línea* del kernel, ejecutada con un depurador de instrucciones (NVIDIA Nsight Compute) [39] antes — y de forma independiente — de la campaña de adquisición de telemetría NVML en tiempo real. Nsight Compute expone con-

tadores de hardware capaces de contabilizar, por cada lanzamiento de kernel, tanto el tráfico total hacia memoria del dispositivo como el número de instrucciones aritméticas de punto flotante efectivamente ejecutadas, discriminadas por tipo de operación (suma, multiplicación y multiplicación–suma fusionada) y por precisión (simple o doble); a partir de estos conteos, la intensidad operacional de la carga se obtiene como el cociente entre el total de operaciones de punto flotante — contando cada multiplicación–suma fusionada como dos operaciones — y el total de bytes transferidos, agregados sobre un número fijo de lanzamientos consecutivos del kernel.

El valor así obtenido es un único número por carga, no una serie por ventana: se declara como una propiedad fija del catálogo, análoga a la suma de verificación del binario, y el post-procesamiento lo copia sin variación a cada una de las muestras NVML de esa corrida. Esto es metodológicamente válido únicamente porque, igual que en CPU, la intensidad operacional de un kernel es una propiedad de su algoritmo y del tamaño del problema que resuelve, no del estado de frecuencia bajo el cual se ejecuta ni del instante de la corrida en que se observa — por lo que no es necesario, ni válido dado el costo de intrusión del perfilado detallado sobre el tiempo de ejecución, repetir esta medición dentro de cada corrida de la campaña principal. Esta vía constituye, igual que en CPU, una medición directa de ambos términos de la ecuación (1.4), aunque agregada por lanzamiento de kernel en vez de por ventana temporal, y calculada una única vez fuera de línea en vez de continuamente durante la ejecución.

Esta discriminación por precisión, sin embargo, presupone que Nsight Compute se invoque solicitando los contadores de **ambas** precisiones simultáneamente para cada carga, y no únicamente los de la precisión que el catálogo declara para ella por diseño de kernel: una auditoría posterior (ARC-110) encontró que las herramientas de caracterización usadas en este trabajo inicialmente seleccionaban un único juego de contadores según esa declaración, de modo que una carga con operaciones reales en la precisión no solicitada las habría subestimado sin ninguna señal de que la mezcla existía. Corregido para solicitar siempre ambas precisiones y contrastar la declaración del catálogo contra lo observado, el método de un único techo Roofline por precisión — el empleado en este trabajo — solo es válido para cargas empíricamente homogéneas en una precisión; una carga con proporciones no triviales de ambas precisiones no admite compararse contra un único ridge sin antes decidir explícitamente cómo tratar esa mezcla (excluirla, tratarla como una categoría aparte, o extender el modelo a varios techos), decisión que este trabajo no adopta unilateralmente para ninguna carga de su catálogo.

Conviene señalar explícitamente por qué el mecanismo de perfilado en GPU no hereda la restricción de presupuesto de contadores físicos simultáneos discutida para CPU en la sección

1.1.3, ni el riesgo de multiplexación asociado a ella. Un contador de PMU de CPU, abierto mediante `perf_event_open`, se muestrea en vivo mientras la carga corre una sola vez; cuando el número de eventos solicitados excede el número de contadores físicos disponibles, el kernel reparte el tiempo de esa única ejecución entre ellos y extrapola cada lectura mediante el factor de escalado tiempo-habilitado/tiempo-contando (sección 1.1.3), un procedimiento que asume una tasa de evento uniforme durante todo el intervalo y que resulta especialmente riesgoso cuando la ventana de observación es corta frente al período de rotación del kernel. Nsight Compute, en cambio, no muestrea en vivo una única ejecución: cuando las métricas solicitadas exceden lo que el hardware de contadores de la GPU puede capturar en una sola pasada, la herramienta *reproduce* el lanzamiento del kernel completo tantas veces como sea necesario, una pasada por cada subconjunto de métricas, bajo la condición de que el kernel sea determinista frente a sus mismas entradas. Cada métrica se obtiene así de una ejecución íntegra del kernel, no de una fracción de tiempo compartida con otras métricas, por lo que no existe una extrapolación lineal equivalente a la de CPU ni una ventana de observación corta que pueda quedar a mitad de una rotación. La precisión aritmética (simple o doble) se discrimina, en consecuencia, sin ningún compromiso de presupuesto: da igual cuántas métricas o precisiones se soliciten, el costo se traduce en más pasadas de perfilado, no en una restricción de cuántas pueden coexistir — un costo asumible precisamente porque, como se explicó arriba, esta medición ocurre una única vez por carga, fuera de la campaña de adquisición de telemetría en tiempo real.

La calibración de los techos del modelo Roofline en GPU sigue el mismo principio general descrito para CPU — microbenchmarks dedicados a saturar por separado cada límite, bajo las condiciones reales de la plataforma —, con dos particularidades propias del dispositivo. Primera, dado que las GPU consideradas ejecutan operaciones de simple y doble precisión a tasas pico sustancialmente distintas, P_{pico} se calibra por separado para cada precisión, y el punto de inflexión de la ecuación (1.3) se calcula igualmente por separado, de modo que una carga expresada en una precisión se compara exclusivamente contra el techo correspondiente a esa misma precisión. Segunda, la carga empleada para calibrar P_{pico} se implementó como un microbenchmark propio de multiplicación–suma fusionada sobre datos residentes en registros, en lugar de recurrir a una biblioteca de álgebra lineal optimizada de terceros: una biblioteca de este tipo puede seleccionar internamente, sin que el usuario lo solicite explícitamente, una ruta de cómputo acelerada por hardware especializado no representativo de la aritmética general de punto flotante del dispositivo, lo que produciría un techo de cómputo inflado frente al alcanzable por las cargas del catálogo que no emplean dicha ruta, y en consecuencia clasificaría erróneamente como *memory-bound* cargas que, medidas contra el techo de aritmética vainilla correspondiente a su precisión, resultan *compute-bound*. Este riesgo es formalmente

equivalente al ya discutido en la sección 2.2.1 para el caso de una carga que cuantifica su trabajo en una unidad distinta de FLOPs: en ambos casos, comparar magnitudes que no están expresadas en el mismo referente invalida la clasificación resultante, aunque el error no se manifieste como una falla explícita del instrumento.

2.3.5. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Puesto que la intensidad operacional de cada carga GPU se fija mediante la medición directa descrita en la sección 2.3.4, cualquier ajuste posterior de los parámetros de ejecución de una carga — por ejemplo, para alcanzar una duración de corrida suficiente para los fines descritos en la sección 2.3.7 — exige distinguir explícitamente entre dos categorías de parámetros según su efecto sobre dicha medición.

La primera categoría comprende los parámetros que repiten el mismo patrón de cómputo un mayor número de veces sin alterar el tamaño del problema resuelto en cada repetición — un número de iteraciones de un mismo esquema numérico, o la duración simulada de un proceso cuyo paso de integración permanece fijo —. Un ajuste de esta naturaleza no puede alterar la intensidad operacional del kernel, dado que cada repetición ejecuta exactamente el mismo balance entre trabajo aritmético y tráfico de memoria que la anterior; en consecuencia, este tipo de ajuste puede aplicarse sin necesidad de repetir la medición de la sección 2.3.4. La segunda categoría comprende los parámetros que alteran el tamaño real del problema — la geometría de una malla, el número de elementos de una simulación de N-cuerpos, o la resolución de una imagen —, para los cuales dicha invarianza no puede asumirse: un problema de mayor tamaño puede modificar la proporción entre trabajo aritmético y tráfico de memoria, entre otras razones porque el conjunto de datos activo deja de residir en un nivel de la jerarquía de caché que sí alcanzaba a contener al problema original, incrementando el tráfico efectivo hacia memoria principal de forma no proporcional al aumento del trabajo aritmético. Por ello, todo ajuste de la segunda categoría se trata como una modificación que invalida la caracterización previa de la carga, y se somete obligatoriamente a una nueva medición mediante el procedimiento de la sección 2.3.4 antes de aceptar cualquier dato adquirido bajo la nueva configuración; cuando dicha remediación confirma un cambio material en la intensidad operacional, se verifica adicionalmente que la carga conserve su clasificación cualitativa frente al punto de inflexión vigente, dado que un cambio de magnitud no implica necesariamente un cambio de régimen.

Cuando una carga no dispone de un parámetro de la primera categoría — por ejemplo, porque su implementación no contempla una noción de repetición interna del mismo cómputo

—, la única vía disponible para extender su duración es un parámetro de la segunda categoría, con el costo de verificación que ello implica; y cuando, adicionalmente, el modelo de ejecución del dispositivo impone un límite estructural sobre dicho parámetro — por ejemplo, un límite documentado sobre el número de unidades de ejecución concurrentes direccionables por un único lanzamiento de kernel —, la duración máxima alcanzable para esa carga queda acotada por dicho límite de hardware, con independencia de cualquier criterio estadístico de suficiencia de muestreo. En tales casos, la imposibilidad de extender la corrida más allá del límite estructural del dispositivo se declara como una limitación del instrumento específica de esa carga, y no se fuerza una extensión que comprometería la validez de la caracterización ya obtenida.

2.3.6. Matriz experimental y control de sesgos

La matriz experimental se define como el producto cartesiano entre las cargas del catálogo, los estados de frecuencia y las repeticiones. Los estados de frecuencia previstos se describen en la Tabla 2.4: además del punto de operación nativo, que sirve como referencia y se obtiene sin intervenir el gobernador del sistema, se contemplan cinco niveles distribuidos uniformemente sobre el rango soportado por la plataforma. Los niveles fijos se expresan como fracciones del rango real detectado en tiempo de ejecución, y no como valores absolutos de frecuencia, con el fin de que la misma definición de campaña sea aplicable a plataformas con rangos distintos.

Tabla 2.4: Estados de frecuencia de la matriz experimental.

Nivel	Definición	Propósito
REF	Gobernador nativo, sin control manual	Línea base de comparación
F0	Extremo superior del rango	Límite de rendimiento y consumo
F1	75 % del rango	Punto intermedio alto
F2	50 % del rango	Punto intermedio central
F3	25 % del rango	Punto intermedio bajo
F4	Extremo inferior del rango	Límite inferior

Para reducir el sesgo experimental se adoptan cuatro medidas. Primera, cada combinación se ejecuta en múltiples repeticiones independientes, entendidas como relanzamientos completos del binario y no como iteraciones internas, de modo que cada repetición parta de un estado de caché y de predictores equivalente. Segunda, el orden de ejecución de las combinaciones se aleatoriza a partir de una semilla registrada en el manifiesto, con el fin de evitar

que una deriva temporal — por ejemplo, térmica — se confunda con el efecto de la carga o de la frecuencia; el registro de la semilla preserva la reproducibilidad del orden. Tercera, las condiciones de afinidad, número de hilos y asignación de núcleos se fijan explícitamente y de forma idéntica para todas las corridas, en lugar de heredarse del entorno. Cuarta, al concluir la campaña se verifica que el estado de frecuencia del sistema haya sido restaurado a su configuración original, y este mismo mecanismo de restauración se registra para ejecutarse también ante una terminación anómala.

2.3.7. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transforman en el conjunto de datos final mediante un procedimiento de diferenciación entre lecturas consecutivas, conforme a lo descrito en la sección 1.1.3. Cada fila resultante representa una ventana de ejecución e incorpora, además de las métricas derivadas, un conjunto de campos de trazabilidad que permiten reconstruir su procedencia: identificador de corrida, carga, nodo, estado de frecuencia solicitado y observado, referencias a los archivos de calibración empleados y suma de verificación del binario ejecutado.

Un elemento central de este procedimiento es que ninguna observación se descarta silenciosamente. Cada ventana recibe una anotación de calidad que describe su condición, según la taxonomía de la Tabla 2.5, y se conserva en el conjunto de datos con dicha anotación. Esta decisión responde a un criterio de transparencia: el filtrado posterior es una decisión del análisis, no del instrumento, y la proporción de ventanas en cada categoría constituye en sí misma un indicador de la calidad de la campaña.

Entre estas categorías, la exclusión por calentamiento delimita explícitamente, para cada carga del catálogo, un intervalo inicial de la corrida declarado como transitorio de arranque, correspondiente al período en que la ejecución aún no alcanza un comportamiento estacionario — por ejemplo, mientras se completan la asignación de memoria, la creación de hilos y la sincronización inicial de la carga. Las ventanas cuya marca temporal cae dentro de ese intervalo se anotan como excluidas por calentamiento y no se emplean en el análisis, de modo que la región efectivamente caracterizada corresponda a la fase repetitiva del kernel y no a su arranque. A diferencia de otras anotaciones de la Tabla 2.5, que se derivan de una condición verificable en la propia ventana, la duración de este intervalo no se fija arbitrariamente por carga ni se estima a partir de una expectativa general de arranque: se determina mediante un procedimiento estadístico aplicado sobre la serie de telemetría de cada corrida, descrito a continuación.

Un aspecto favorable para la aplicación de este procedimiento es que la duración del ca-

lentamiento nunca condiciona la adquisición: el intervalo declarado se emplea exclusivamente como criterio de filtrado durante el post-procesamiento, y no se comunica en ningún momento al ejecutor de la capa C++, de modo que toda corrida ya adquirida contiene, desde el primer instante, la totalidad del transitorio de arranque real, con independencia del valor de calentamiento vigente al momento de adquirirla. Esta propiedad permite recalcular y ajustar el criterio de calentamiento de forma retroactiva sobre corridas ya realizadas, sin necesidad de una nueva adquisición.

El procedimiento se desarrolla en dos etapas, aplicadas sobre una señal representativa del estado de carga del dispositivo — instrucciones por ciclo en el caso de CPU, utilización del dispositivo reportada por la interfaz de gestión de GPU en el caso de GPU —, calculada sobre ventanas móviles consecutivas de tamaño fijo.

En la primera etapa se aplica un criterio de estabilidad basado en el coeficiente de variación (CV %) de dicha señal, siguiendo el principio de evaluación estadísticamente rigurosa de desempeño propuesto por Georges et al. [37]: se declara alcanzado el fin del transitorio de arranque en el primer instante a partir del cual el CV % se mantiene por debajo de un umbral en dos ventanas consecutivas — exigencia de dos ventanas, y no de una sola, para no confundir el fin del arranque con una fluctuación transitoria dentro de una fase todavía inestable, ni con un cambio de fase legítimo del propio kernel que ocurra más adelante en la corrida —, y al punto detectado se le aplica un margen de seguridad multiplicativo antes de declararlo como intervalo de calentamiento definitivo. Una limitación de este criterio, identificada durante su verificación empírica sobre la señal de utilización de GPU, merece señalarse explícitamente: dado que el coeficiente de variación se define como el cociente entre la desviación estándar y la media, una ventana cuya señal permanece en cero satisface trivialmente el umbral, de modo que el criterio, aplicado sin más, puede declarar estable el reposo inicial del dispositivo — previo a que exista actividad real — en lugar de la fase de carga sostenida que se pretende delimitar, precisamente el resultado opuesto al buscado. Este modo de falla se corrige exigiendo, adicionalmente al umbral de CV %, que la media de la ventana supere un piso mínimo de actividad antes de aceptar la ventana como estable.

En la segunda etapa, reservada para las cargas donde el criterio de CV % no logra identificar ningún punto que satisfaga la condición anterior — un modo de falla conocido de este tipo de heurística de umbral fijo —, se aplica como respaldo un método de detección de puntos de cambio (*change point detection*) sobre la misma señal: en lugar de asumir que el estado estable es plano dentro de un umbral, el método busca recursivamente la partición de la serie que más reduce la suma de errores cuadráticos dentro de cada segmento resultante, en la línea de los métodos empleados para caracterizar estadísticamente el comportamiento

de arranque de máquinas virtuales [38]. Cuando la serie presenta más de un punto de cambio — por ejemplo, una fluctuación breve y espuria previa al inicio real de la carga sostenida —, se descarta el criterio de aceptar sin más el primer punto detectado, y se selecciona en su lugar el primer segmento cuya media alcanza una fracción declarada de la meseta de mayor carga observada en toda la corrida, de modo que una fluctuación que no llega a sostenerse no se confunda con el fin genuino del arranque.

Como verificación final, antes de aceptar el resultado de cualquiera de las dos etapas, la transición detectada se contrasta contra una señal de referencia independiente de la empleada para la detección — la potencia reportada por el dispositivo —: una transición genuina de régimen de reposo a régimen de carga sostenida debe manifestarse también como un salto identificable en la potencia consumida, y no únicamente en la señal primaria de detección. Cuando ninguna de las dos etapas anteriores encuentra, en la señal de potencia, una transición identificable a lo largo de toda la corrida — situación observada en cargas cuyas unidades individuales de trabajo resultan demasiado breves para que la cadencia de muestreo del dispositivo resuelva su actividad, con independencia de cuánto se alargue la duración total de la corrida conforme al criterio de la sección 2.3.5 —, se concluye que la señal disponible no sostiene una medición confiable del intervalo de calentamiento para esa carga específica. En ese caso, el intervalo se conserva en su valor por defecto y la imposibilidad de medirlo se documenta explícitamente como evidencia negativa, en lugar de forzar un valor no respaldado por la medición.

Tabla 2.5: Taxonomía de anotaciones de calidad por ventana de muestreo.

Anotación	Condición
Válida	La ventana satisface todos los criterios de calidad.
Sin diferencia previa	Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.
Excluida por calentamiento	La ventana pertenece al intervalo inicial declarado como transitorio de arranque.
Contadores degradados	Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.
Intensidad indefinida	No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable.
Energía inválida	La lectura energética correspondiente no es válida en una campaña que la requiere.
Sin lectura de frecuencia	No se dispone de la frecuencia efectivamente observada durante la ventana.

De forma complementaria, cada corrida completa se somete a una validación posterior que examina su integridad global: ausencia de pérdidas en la estructura de muestreo, correspondencia de la suma de verificación del binario, cumplimiento del criterio de éxito propio de la carga y unicidad del identificador de corrida. Una corrida que no supere esta validación se registra como rechazada, conservando sus artefactos para inspección, en lugar de eliminarse.

2.3.8. Medición directa de FLOPs por ventana y su validación empírica

El diseño original de este instrumento estimaba los FLOPs de cada ventana mediante un prorrateo del total reportado por el propio programa, en proporción a las instrucciones retiradas de cada ventana, bajo el supuesto de que la densidad de operaciones de punto flotante por instrucción retirada se mantiene aproximadamente constante dentro de una misma corrida. Dicho supuesto no se dio por válido sin verificación. Antes de adoptarlo como definitivo, se evaluó si la plataforma experimental permitía sustituirlo por una medición directa, siguiendo el mismo criterio de no asumir capacidades del hardware sin confirmarlas empíricamente que rige el resto de este instrumento (secciones 1.1.5 y 1.1.3). Esta sección documenta ese procedimiento de verificación y su resultado; el prorrateo original, descartado tras esta validación, queda documentado como antecedente metodológico en el registro de cambios del

proyecto (ARC-27.1), no en el instrumento.

Encoding y ponderación. Para la plataforma experimental (Intel Xeon Gold 5315Y, Ice Lake-SP), la familia de eventos `FP_ARITH_INST_RETIRED` utiliza el *EventSelect* `0xC7`; cada modalidad aritmética se distingue mediante su *Unit Mask* (UMask), de modo que el encoding completo de cada sub-evento no se reduce a `0xC7` por sí solo. Los cuatro sub-eventos de doble precisión empleados se distinguen por el ancho del registro vectorial — escalar, y empaquetado de 128, 256 y 512 bits — mediante los UMask `0x01`, `0x04`, `0x10` y `0x40` respectivamente, sin alias simbólico expuesto por el kernel de Linux en este nodo, por lo que se programan como evento crudo mediante `perf_event_open`. Estos eventos contabilizan instrucciones retiradas, no FLOPs de forma abstracta: convertir su conteo requiere el número de elementos de doble precisión por instrucción (1, 2, 4 y 8 para escalar/128/256/512 bits) y el tipo de operación. Las instrucciones *fused multiply-add* (FMA) no exigen una ponderación adicional: la propia definición del evento hace que una FMA incremente el contador en 2 por elemento en vez de 1, precisamente para que la cuenta ya represente FLOPs y no instrucciones en bruto [33] (Tabla 2.6). El total de FLOPs medidos en una ventana se obtiene entonces ponderando cada sub-evento únicamente por su número de elementos:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta N_{\text{escalar},i} + 2 \cdot \Delta N_{128,i} + 4 \cdot \Delta N_{256,i} + 8 \cdot \Delta N_{512,i} \quad (2.1)$$

Tabla 2.6: FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento `FP_ARITH_INST_RETIRED` de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.

Ancho vectorial	Elementos FP64	FLOPs (ADD/MUL)	FLOPs (FMA)
Escalar	1	1	2
128 bits	2	2	4
256 bits	4	4	8
512 bits	8	8	16

El encoding se determinó a partir de la definición oficial de Intel para esta microarquitectura [33], corroborada contra la tabla de eventos de LIKWID, que documenta el grupo `FLOPS_DP` con esta misma ponderación para Ice Lake-SP [29], en lugar de inferirse a partir de documentación genérica de otras microarquitecturas.

Diseño de la verificación. Se construyó un instrumento de prueba independiente del harness de producción, con el mismo protocolo de atribución por PID e `inherit=1` descrito en la sección 1.1.5, capaz de abrir simultáneamente los diez contadores que componían el

presupuesto inicial: los seis entonces establecidos (sección 1.1.3) más los cuatro sub-eventos de `FP_ARITH_INST_RETIRED` de la ecuación (2.1). Este instrumento se ejecutó contra dos kernels de régimen opuesto del catálogo experimental: uno intensivo en cómputo (multiplicación de matrices densas, con un volumen de trabajo conocido analíticamente) y uno intensivo en memoria (un kernel multi-hilo de la suite NAS Parallel Benchmarks), en ambos casos bajo la afinidad de núcleos real de la campaña.

Resultados. Para el kernel de cómputo denso, cuyo total de FLOPs se conoce exactamente por construcción ($2 \cdot \text{iteraciones} \cdot n^3$), la ecuación (2.1) reprodujo ese total con un error relativo de 0.29 % sin afinidad fijada y de 0.30 % bajo la afinidad real de campaña. Para el kernel intensivo en memoria, el contraste se hizo contra el total de FLOPs que el propio kernel reporta al finalizar, arrojando un error relativo de 7.48 %; este valor es mayor que el del primer kernel, pero es consistente con una causa identificada y ajena al contador: el tiempo que dicho kernel cronometra internamente cubre únicamente su ciclo de cómputo principal y excluye la fase final de verificación del resultado, la cual también ejecuta operaciones de punto flotante que el contador de hardware sí registra por abarcar el proceso completo. En ambos casos, los diez contadores abrieron simultáneamente sin evidencia de multiplexación (razón tiempo-contando sobre tiempo-habilitado igual a 1 en los diez), confirmando que esa combinación era sostenible bajo el estado del nodo observado en aquella verificación. Una prueba de humo posterior encontró que la activación del vigilante NMI del sistema operativo había reducido el presupuesto efectivo; la configuración de producción se ajustó entonces a nueve eventos retirando `L2_LINES_IN_ALL`, sin eliminar ninguno de los cuatro sub-eventos que sostienen la medición de FLOPs (sección 3.5.1).

Confirmación a escala de campaña. Con la verificación anterior favorable, la medición directa se integró al harness de producción como única fuente de FLOPs por ventana — sin conservar el prorrato original como respaldo, una vez confirmado que no aportaba nada en la plataforma y el alcance de este trabajo (sección 1.1.2) —, y se ejecutó una campaña real completa: siete kernels del catálogo, matriz declarada a seis estados de frecuencia (REF y F0-F4), tres repeticiones por combinación, 66 corridas aceptadas o rechazadas por los criterios de la sección 2.3.7 y siguientes. Al momento de esta campaña, el nodo aún no tenía concedido el permiso administrativo de escritura de frecuencia, de modo que los seis niveles se ejecutaron en la práctica sobre la frecuencia nativa del procesador; esto no compromete la validación de la medición de FLOPs, que ocurre por ventana con independencia del estado de frecuencia, pero significa que esta campaña específica no debe citarse como evidencia de que el control de frecuencia (DVFS) en sí mismo fue ejercitado. Sobre aproximadamente 1.29 millones de

ventanas con delta válido en esa campaña, el 100 % obtuvo su valor de FLOPs mediante la ecuación (2.1), y ninguna ventana resultó con los contadores de punto flotante degradados. Este resultado, a una escala varios órdenes de magnitud mayor que la verificación inicial de dos kernels, respalda que la disponibilidad y estabilidad del encoding no es un caso aislado favorable, sino una propiedad sostenida de la combinación plataforma-instrumento descrita en este capítulo.

Alcance de esta validación. Los resultados anteriores se verificaron específicamente para la microarquitectura Ice Lake-SP de la plataforma experimental (Tabla 2.1), gateados por familia y modelo de procesador en el mismo mecanismo que protege los demás eventos crudos de este instrumento (sección 1.1.3); no se extrapolan a otro hardware sin repetir este procedimiento. Tampoco se reclama que el error relativo observado (0.29–7.48 %) sea una cota universal: ambos valores dependen del kernel evaluado y, en el segundo caso, de una particularidad de cómo ese kernel específico delimita su propio tiempo de ejecución. Dado que este trabajo no reclama portabilidad más allá de la plataforma descrita, y que la medición directa estuvo disponible sin excepción en las 66 corridas de la campaña real, el instrumento no conserva ningún mecanismo de respaldo para un nodo hipotético donde este encoding no se haya validado: una ventana en esa situación queda con intensidad operacional indefinida, no con un valor estimado.

2.3.9. Reproducibilidad

Toda campaña queda definida por un manifiesto declarativo bajo control de versiones, que especifica las cargas, los estados de frecuencia, el número de repeticiones, el período de muestreo, la asignación de núcleos, la semilla de aleatorización y las referencias de calibración. Junto con los datos, cada campaña persiste el perfil de capacidades detectado en el nodo, los resultados de calibración y los metadatos de cada corrida. El instrumento de medición, el orquestador y las definiciones de campaña se mantienen en un repositorio público, de modo que la campaña sea reproducible a partir de los artefactos publicados.

Como verificación adicional de consistencia, toda vez que se modifica un parámetro de ejecución de una carga del catálogo — por ejemplo, a raíz de un ajuste realizado conforme a lo descrito en la sección 2.3.5 — se ejecuta una corrida completa de la campaña que incluya dicha carga, contrastando que la etiqueta derivada, la intensidad operacional y el punto de inflexión empleado resulten idénticos entre las distintas repeticiones independientes de esa misma carga. Esta comparación no sustituye la verificación de calidad por ventana descrita en la Tabla 2.5, sino que actúa como una verificación de nivel superior: una discrepancia

entre repeticiones nominalmente idénticas indicaría una fuente de no determinismo en el instrumento o en la propia carga, y no únicamente un artefacto puntual de una corrida concreta.

2.4. Fase 2: Desarrollo del modelo de aprendizaje automático

En esta fase se desarrollan las técnicas de análisis orientadas a la construcción de un modelo predictivo capaz de identificar el comportamiento de la carga de trabajo en tiempo de ejecución. Inicialmente se realizará un proceso de preprocesamiento que incluye la limpieza, la normalización y la selección de características relevantes, partiendo del conjunto de datos producido en la Fase 1 y aplicando sobre él los criterios de filtrado que se estimen adecuados a partir de las anotaciones de calidad descritas en la Tabla 2.5.

El problema se formula como una tarea de clasificación supervisada binaria, donde el modelo recibe como entrada un vector de telemetría correspondiente a una ventana de ejecución y produce como salida la etiqueta de régimen. Dado que el sistema debe operar en tiempo de ejecución, se prioriza el uso de modelos de baja complejidad computacional, entre ellos árboles de decisión, bosques aleatorios, regresión logística y ensambles potenciados. Durante esta fase se entrenarán y evaluarán distintos modelos candidatos, comparados mediante métricas de clasificación y mediante la latencia de inferencia medida.

Un criterio metodológico relevante en esta fase es la estrategia de partición del conjunto de datos. Dado que las ventanas provenientes de una misma corrida no son observaciones independientes entre sí, una partición aleatoria a nivel de ventana produciría una estimación optimista de la capacidad de generalización, al permitir que ventanas contiguas de la misma ejecución aparezcan simultáneamente en entrenamiento y prueba. Por ello, la partición se realizará a nivel de corrida o de carga, de modo que la evaluación mida la capacidad del modelo para generalizar hacia ejecuciones no observadas. A partir de este análisis se seleccionará el modelo que represente el mejor compromiso entre desempeño predictivo y costo de inferencia, y se serializará para su integración en el sistema de control.

2.5. Fase 3: Implementación del agente de control DVFS

La tercera fase corresponde al desarrollo del agente de control en espacio de usuario, materializado como un servicio en segundo plano encargado de monitorear el estado del hardware y aplicar decisiones de control basadas en el modelo entrenado.

El funcionamiento se basa en un ciclo iterativo en el cual se capturan las métricas actuales del sistema, se construye el vector de entrada del modelo, se ejecuta la inferencia y, a partir de la predicción, se determina y aplica la configuración de frecuencia. La política de control será discreta: no resolverá un problema continuo de optimización, sino que seleccionará un estado dentro del conjunto de configuraciones soportadas por el hardware. La decisión se tomará de forma proactiva, guiada por el régimen inferido y no únicamente por métricas reactivas de utilización.

El diseño de esta fase incorpora dos consideraciones derivadas del marco conceptual. La primera es que el mecanismo de actuación debe adaptarse al controlador de frecuencia activo en la plataforma, dado que la vía para fijar un punto operativo difiere según se trate de un controlador que expone un gobernador de espacio de usuario o de uno que solo admite la restricción del rango permitido. La segunda es que la frecuencia de decisión debe acotarse considerando la latencia de conmutación: una política que solicite cambios a un ritmo superior al que el sistema puede materializar no solo no obtendrá el beneficio esperado, sino que incurrirá en el costo de las transiciones sin amortizarlo [18, 27]. En consecuencia, el período de decisión del agente se tratará como un parámetro de diseño a determinar empíricamente, y no como una constante fijada a priori.

2.6. Fase 4: Validación experimental y análisis de resultados

La fase final evalúa empíricamente el impacto del sistema propuesto sobre el consumo energético y el rendimiento, en concordancia con el cuarto objetivo específico. Se ejecutarán las cargas seleccionadas bajo escenarios de referencia: gobernador nativo del sistema operativo, configuración de frecuencia fija de alto rendimiento, y ejecución orquestada por el agente propuesto. En cada ejecución se medirán el tiempo total de ejecución, la energía consumida, la potencia media, el EDP y el *overhead* atribuible al agente.

Dado que el objetivo no es únicamente reportar métricas sino establecer si las diferencias observadas son estadísticamente defendibles, los resultados se analizarán mediante técnicas estadísticas seleccionadas según la distribución y la homogeneidad de varianzas de los datos, reportando medidas de dispersión y tamaños de efecto junto con las pruebas de significancia.

2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda dicha asignación, de acuerdo con lo expuesto en la sección 1.1.6. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos. Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

Capítulo 3

Resultados

Este capítulo reporta los resultados de la Fase 1: la construcción y validación del instrumento de medición, la campaña CPU solicitada en múltiples niveles — junto con la verificación posterior que demostró que estos no constituyeron frecuencias físicas distintas — y la caracterización de precisión aritmética del catálogo de GPU. Las Fases 2–4 — entrenamiento del clasificador, implementación del agente y evaluación de EDP frente a los gobernadores nativos — están fuera del alcance de estos resultados y se reportarán al cerrarse el desarrollo completo del proyecto.

3.1. Descripción del conjunto de datos experimental

El catálogo experimental final está compuesto por 9 cargas de CPU (7 de conjunto de datos y 2 de calibración) y 12 cargas de GPU (8 de conjunto de datos y 4 de calibración). Todas las cargas de CPU operan en doble precisión, verificado empíricamente para las 9 sobre el catálogo completo, no solo sobre las empleadas en la validación principal (sección 2.3.8). El conjunto de datos GPU combina siete kernels de la suite Rodinia y una multiplicación densa de matrices sobre cuBLAS. Las cuatro cargas de calibración corresponden a tres microbenchmarks propios que sostienen los techos Roofline — ancho de banda, pico FP32 y pico FP64 — y una DGEMM de cuBLAS conservada como referencia informativa independiente.

La composición de GPU no es la originalmente propuesta: una auditoría de precisión aritmética — descrita en la sección 3.3 — encontró que uno de los siete kernels de Rodinia inicialmente incluidos ejecutaba una fracción no trivial de su trabajo en una precisión distinta de la declarada, con una intensidad operacional resultante demasiado cercana al punto de inflexión como para clasificarlo con confianza; se retiró del catálogo y se reemplazó por una

eliminación gaussiana de la misma suite, confirmada empíricamente pura en la precisión declarada.

3.2. Validación de la actuación DVFS en CPU

Con el permiso administrativo de escritura de frecuencia concedido durante el desarrollo de este trabajo, se ejecutó una campaña completa sobre la plataforma descrita en la Tabla 2.1: 7 kernels de conjunto de datos, 6 niveles solicitados (REF y F0–F4, Tabla 2.4), 3 repeticiones por combinación — 126 combinaciones en total. La corrida limpia de reemplazo se aceptó íntegramente a nivel del instrumento: 126 de 126 combinaciones aceptadas, ninguna rechazada ni reutilizada de intentos anteriores, 4.67 horas-núcleo consumidas, 1,355,352 ventanas producidas (95.4 % con estado de calidad ok) y restauración final verificada. Esta aceptación confirma la integridad de la adquisición y del etiquetado, pero no basta para afirmar que los seis niveles solicitados correspondieron a seis frecuencias físicas distintas.

La escritura y su relectura inmediata sí funcionaron: para cada nivel fijo, `scaling_max_freq` y `scaling_min_freq` conservaron exactamente el valor solicitado — 3.6, 2.9, 2.2, 1.5 y 0.8 GHz para F0–F4, respectivamente —, y la restauración devolvió el sistema a su estado anterior. Sin embargo, la columna que inicialmente se interpretó como frecuencia observada durante la carga provenía de una sola lectura de `scaling_cur_freq` realizada después de terminar el proceso y difundida a todas sus ventanas; por tanto, no verificaba el reloj sostenido durante la ejecución. El instrumento se corrigió incorporando un lector de solo lectura en la capa C++, muestreado en el mismo ciclo del productor que los contadores de PMU. Con esa medición ya temporalmente correcta, una carga solicitada a 2.2 GHz mostró aproximadamente 3.5 GHz durante la ejecución. Una comprobación independiente, realizada fuera del instrumento mediante lecturas periódicas de sysfs mientras se ejecutaba otro kernel, reprodujo el resultado en 40 lecturas consecutivas.

La causa observable es que la plataforma mantiene activo el turbo bajo `intel_pstate/HWP`, mientras los controles globales capaces de limitarlo (`no_turbo` y `max_perf_pct`) no son escribibles con los permisos actuales. En estas condiciones, aceptar una escritura en los límites por núcleo demuestra que el kernel almacenó la solicitud, no que el hardware la respetó bajo carga. En consecuencia, la campaña no se presenta como un barrido DVFS válido ni como evidencia del efecto de la frecuencia sobre tiempo, energía o EDP. Sus intensidades y etiquetas siguen describiendo las cargas en el punto de operación realmente observado, pero las repeticiones F0–F4 no pueden tratarse como estados DVFS independientes hasta resolver el control de turbo y repetir la adquisición con la nueva verificación por ventana.

Alcanzar este resultado exigió corregir tres fallas reales del instrumento, encontradas únicamente al ejercitar por primera vez una campaña multi-frecuencia densa y de larga duración contra hardware real — un tipo de falla que ninguna prueba unitaria ni una campaña corta puede exponer por construcción. Primero, el chequeo de carga externa (una de las verificaciones previas de la sección 2.3.1) comparaba el promedio de carga del sistema completo contra un umbral fijo, sin distinguir el trabajo legítimo de la propia campaña sobre sus núcleos delegados de una contaminación real; en una campaña de una hora de duración sobre 6 núcleos dedicados, la media exponencial de 1 minuto del sistema operativo converge naturalmente cerca de ese mismo número, y el chequeo rechazó 105 de 126 combinaciones por una razón que resultó ser su propio trabajo esperado. Corregido para comparar el exceso sobre el número de núcleos delegados, no el promedio total. Segundo, la verificación de escritura de frecuencia de CPU falló de forma intermitente al fijar el nivel más alto del rango inmediatamente después de una calibración de pico de cómputo que satura los núcleos delegados al 100 %, reproducible solo dentro del harness y nunca en una escritura equivalente fuera de él — consistente con que el gestor de energía del procesador rechace transitoriamente una escritura bajo carga activa; corregido con un reintento acotado. Tercero, un reconfigure completo del sistema de construcción, necesario para incorporar los objetivos de prueba de GPU, restableció silenciosamente el compilador por defecto del nodo a uno cuya biblioteca de tiempo de ejecución resultó incompatible con el binario del harness, impidiendo que arrancara en absoluto; corregido fijando explícitamente el compilador de construcción.

La secuencia experimental deja, por tanto, dos conjuntos con alcances distintos. La primera campaña de 126 combinaciones quedó descartada para clasificación porque precede tanto a la medición directa de bytes de *uncore* (sección 3.5) como a la corrección del punto de inflexión Roofline (sección 3.6). La campaña limpia de reemplazo ya incorpora ambas correcciones y el ajuste del presupuesto de contadores descrito en la sección 3.5.1; sus datos de clasificación son utilizables dentro del único régimen de frecuencia efectiva observado, pero su eje F0–F4 queda invalidado por el hallazgo posterior de turbo. Una nueva campaña solo tendrá valor como conjunto DVFS después de resolver esa actuación y comprobarla durante cada carga con el lector por ventana ya implementado.

3.3. Caracterización de precisión aritmética en el catálogo de GPU

Como se argumentó en la sección 2.3.4, la intensidad operacional de cada carga de GPU se determina mediante perfilado con Nsight Compute, discriminado por precisión aritmética.

Las herramientas empleadas originalmente para esa caracterización seleccionaban los contadores de una única precisión — la que el catálogo declaraba para cada kernel por diseño —, sin verificar si el kernel ejecutaba, además, una fracción real de trabajo en la precisión no solicitada. Se corrigió el procedimiento para solicitar siempre ambas precisiones simultáneamente y se reperfiló el catálogo completo de siete kernels de Rodinia, contrastando la declaración original contra lo observado.

Tres kernels resultaron empíricamente puros en la precisión declarada, sin cambios: una simulación de N-cuerpos (100 % doble precisión, intensidad operacional observada dentro de 0.1 % del valor previamente declarado), una descomposición LU (100 % simple precisión, con convergencia verificada hasta 200 lanzamientos de kernel) y una transformada de *wavelets* (100 % simple precisión, dentro de 0.5 % del valor previamente declarado).

Los otros cuatro kernels mostraron una mezcla de precisión no trivial. Un kernel de simulación térmica ejecutaba 30.4 % de sus operaciones en doble precisión pese a declararse de precisión simple, con una intensidad operacional real de 7.21 FLOP/byte — justo en el borde del punto de inflexión de simple precisión (7.28–7.98 FLOP/byte según la calibración) —, sin base metodológica sólida para compararlo contra un único techo sin antes decidir cómo tratar la mezcla; se retiró del catálogo en lugar de forzar una clasificación no sustentada. Dos kernels adicionales (retropropagación y un solucionador de ecuaciones diferenciales ordinarias) resultaron mayoritariamente de doble precisión (82.3 % y 71.3 % respectivamente) pese a declararse de precisión simple; en ambos casos la clasificación *memory-bound* no cambió — la intensidad operacional real permanece órdenes de magnitud por debajo de cualquier techo posible en esta plataforma —, pero la precisión declarada en el catálogo se corrigió para reflejar lo medido.

El cuarto caso corresponde a la multiplicación densa de matrices sobre cuBLAS, cuya ruta de ejecución ya se había confirmado acelerada por unidades Tensor Core en un trabajo previo de este proyecto. Se verificó adicionalmente que su intensidad operacional declarada (68.0 FLOP/byte) no proviene del mismo método de conteo de instrucciones que los kernels de Rodinia — el cual, aplicado a esta ruta de ejecución, habría producido un valor cuatro órdenes de magnitud menor, dado que las instrucciones Tensor Core no incrementan los contadores clásicos de multiplicación–suma fusionada —, sino del conteo analítico del propio algoritmo, contrastado independientemente contra la tasa que el binario reporta al finalizar, con una discrepancia menor a 0.2 % entre ambos métodos. El valor permanece correcto; solo su procedencia documentada era imprecisa.

3.4. Estado del control de frecuencia en GPU

La actuación de frecuencia de GPU se implementa mediante el bloqueo de reloj de *streaming multiprocessor* (`nvidia-smi -lgc`), delegado mediante privilegios restringidos de `sudo`. Aunque este era ya el mecanismo operativo previsto por la plataforma, era necesario confirmar de forma independiente que el objetivo se sostuviera durante una carga real y no solo que el comando terminara con éxito.

Un diagnóstico anterior, ejecutado con NVIDIA 595.45.04/CUDA 13.2, observó 765 MHz pese a solicitar cinco objetivos mediante distintas sintaxis y repetir la prueba con dos kernels. La telemetría consultada no mostró una explicación térmica, de potencia o de gestión. Ese resultado motivó verificaciones adicionales, pero no se toma aquí como demostración de que `-lgc` estuviera averiado o indisponible: no se conservó una comparación controlada que permita reconciliar aquella observación con la actuación confirmada posteriormente, y el mecanismo de bloqueo ya funcionaba en la plataforma. Se mantiene únicamente como antecedente de por qué un código de retorno exitoso no se consideró evidencia suficiente por sí solo.

La verificación concluyente se realizó dentro de una asignación exclusiva, con un microbenchmark FP64 propio como carga y restauración automática del estado al finalizar. Un objetivo de 600 MHz produjo 305 lecturas con utilización activa, todas exactamente a 600 MHz; un segundo objetivo de 1200 MHz produjo 174 lecturas activas, todas exactamente a 1200 MHz. Por tanto, el actuador retiene el reloj solicitado bajo carga real en los dos niveles probados. La plataforma exponía NVIDIA 610.57.04/CUDA 13.3 durante esta prueba; esas versiones se registran para reproducibilidad, sin inferir que hayan originado la funcionalidad ni que exista una transición causal respecto del diagnóstico anterior.

En paralelo se completó el *plumbing* que una campaña DVFS de GPU exige: verificación de actividad ajena antes de cada combinación, piso mínimo de utilización de 5 % para que una muestra supere el ruido del sensor, y desacoplamiento completo de los ejes de frecuencia CPU y GPU mediante su producto cartesiano. Estas piezas cuentan con pruebas unitarias y el actuador ya fue verificado en hardware en dos niveles, pero el producto cartesiano y el pipeline completo multi-frecuencia aún no se han ejercitado como una campaña GPU de producción. No existe un bloqueo conocido de actuación GPU; la recolección y aceptación del conjunto multi-frecuencia permanecen como el siguiente paso experimental.

3.5. Medición directa de bytes movidos en memoria (*uncore*)

Un segundo permiso administrativo, otorgado durante el desarrollo de este trabajo, instaló la herramienta **perf** en la plataforma y le asignó la capacidad **CAP_PERFMON** como propiedad del propio binario, con el fin de habilitar la lectura de los contadores de *uncore* del controlador de memoria (sección 1.1.3) sin reducir la restricción de **perf_event Paranoid** para todo el sistema. Se construyó un instrumento completo para aprovecharlo: dado que una capacidad asignada a un archivo solo se activa para el proceso que ejecuta exactamente ese archivo — no para cualquier proceso que abra el mismo tipo de evento por su cuenta, algo confirmado empíricamente antes de descartar el primer diseño, ya que el proceso propio del instrumento reportaba capacidades efectivas vacías pese a que **CAP_PERFMON** figuraba entre las disponibles del sistema —, el diseño final invoca a **perf** como subproceso y procesa en vivo su salida periódica, en lugar de abrir los contadores directamente.

El instrumento se implementó y validó por completo del lado del proyecto: verificado con pruebas unitarias sobre el análisis de su formato de salida (incluidos los casos de un contador rechazado y de un separador de campo que evita colisionar con la propia sintaxis de eventos crudos, que contiene comas) y sobre la degradación controlada ante la ausencia de la herramienta. Ejecutado directamente sobre la plataforma real, sin embargo, el propio binario **perf** — no ya el instrumento de este proyecto — reportó no tener la capacidad efectivamente asignada: la consulta estándar del sistema operativo sobre las capacidades de archivo del binario no devolvió ninguna, y al solicitarle explícitamente contar en modo de sistema, **perf** devolvió el mismo mensaje de error que documenta la ausencia de **CAP_PERFMON**. El hallazgo, con la evidencia reproducida directamente sobre el nodo, se reportó a la administración del clúster, que corrigió la asignación de la capacidad sobre el binario correspondiente en la plataforma real. Verificado nuevamente tras la corrección: la consulta estándar de capacidades de archivo ya devuelve **cap_perfmon** sobre el binario, y una lectura directa de un contador de *uncore* bajo carga real reporta una cuenta válida en lugar del error de capacidad.

La medición directa de *uncore* reemplaza por completo al *proxy* de fallos de caché en la clasificación por ventana — no como respaldo, sino como única fuente admitida — aunque no a la granularidad fina de una ventana individual: **perf stat** reporta en intervalos de al menos ~ 10 ms, mientras que una ventana de CPU dura ~ 1 ms, así que asignar el conteo de bytes de un intervalo completo a una sola ventana angosta dividiría un numerador de FLOPs de escala fina contra un denominador de bytes de escala gruesa, un error de consistencia física, no solo de precisión. El instrumento agrega los FLOPs de todas las ventanas cubiertas por cada

intervalo de *uncore* y calcula la intensidad operacional real a esa granularidad más gruesa, difundida a cada una de esas ventanas como `operational_intensity/phase_label_train`. Una ventana que ningún intervalo real llega a cubrir — el arranque de la corrida, o una corrida sin esta medición habilitada — queda deliberadamente indefinida, en vez de recurrir al *proxy* como alternativa: mezclar, incluso solo en algunas filas, una medición real con una conocida por subestimar el tráfico de precarga de hardware volvería inconsistente la calidad de la propia columna de clasificación fila a fila, sin una señal visible de cuál fue cuál más allá de columnas auxiliares. `bytes_moved_window` (el *proxy*) se sigue calculando y reportando en su propia columna, únicamente como referencia de comparación — nunca alimenta la clasificación, ni siquiera de respaldo.

Verificado de extremo a extremo con una carga de tráfico de memoria real: de 332 ventanas de una corrida de referencia, 322 quedaron cubiertas por al menos un intervalo real de *uncore*, con conteos de lectura/escritura variables y coherentes entre intervalos consecutivos — nunca en cero, la señal esperada de una medición genuina bajo carga activa. El diseño de agregación se confirmó directamente: el mismo intervalo de *uncore* cubrió varias ventanas consecutivas, y todas recibieron idéntico valor de bytes difundido, nunca fraccionado entre ellas. De esas 322, 301 quedaron efectivamente clasificadas con el valor real; las 21 restantes tenían cobertura de bytes pero al menos una ventana del grupo sin FLOPs medidos válidos, y quedaron correctamente indefinidas en vez de calcular una intensidad con un numerador incompleto — confirmado explícitamente contrastando esas 21 y las ventanas sin ningún intervalo que las cubriera (la primera de la corrida y las del tramo final) contra la fórmula del *proxy*: el valor que habría producido coincidía, cifra por cifra, con el que quedó descartado. La primera campaña de CPU ya aceptada se había recolectado antes de esta corrección y quedó descartada para clasificación; la campaña limpia de reemplazo descrita en la sección 3.2 sí incorpora esta señal en todas sus corridas.

Antes de comprometer una campaña real de varias horas a este instrumento, se ejecutó una prueba de esfuerzo dirigida a las condiciones que una corrida aislada no expone: cinco repeticiones consecutivas del mismo patrón de arranque y cierre del subproceso que una campaña real repite en cada corrida, una carga de duración extrema por debajo (microsegundos) del piso práctico de intervalo de la medición, y una carga más prolongada para observar la cadencia de intervalos de principio a fin. Las cinco repeticiones consecutivas completaron con datos reales y sin dejar ningún proceso remanente del subproceso de medición al finalizar; la carga extremadamente corta se degradó de forma controlada, sin fallo ni bloqueo, con apenas dos intervalos correspondientes al arranque de la medición antes de que la carga siquiera comenzara.

La prueba con la carga más prolongada expuso un defecto real, no de exactitud numérica sino de cobertura: los primeros intervalos de cada corrida quedaban agrupados entre sí por microsegundos — muy por debajo del piso real de ~ 10 ms — en lugar de mostrar el espaciado uniforme del resto de la corrida. La causa fue que el instrumento marcaba cada intervalo con la hora del reloj del propio proceso en el momento en que lo procesaba, no con la hora real en que el subproceso de medición cerró ese intervalo; durante la espera inicial de confirmación de que el subproceso arrancó correctamente, este ya había emitido varios intervalos que quedaban pendientes de leer, y al leerlos todos de una vez se les asignaba, a cada uno, una marca de tiempo casi idéntica a la de sus vecinos. El efecto no alteraba la cantidad de bytes contada en cada intervalo, pero sí reducía cuántas ventanas de CPU lograban emparejarse con alguno de ellos durante los primeros instantes de cada corrida, al quedar los intervalos agrupados en una franja de tiempo demasiado angosta para que ninguna ventana cayera dentro de ella. Corregido ligando la marca de tiempo de cada intervalo al reloj relativo que el propio subproceso de medición ya reporta en su salida, anclado una sola vez al instante real de su arranque, en vez de al momento en que el instrumento tuvo oportunidad de procesarlo. Verificado nuevamente sobre la misma prueba: el espaciado mínimo entre intervalos consecutivos pasó de fracciones de microsegundo a los ~ 10 ms esperados, de manera consistente a lo largo de toda la corrida, incluidos sus primeros instantes. Las tres pruebas de esfuerzo se repitieron íntegramente después de esta corrección, con el mismo resultado limpio de antes más el espaciado corregido, antes de dar el instrumento por listo para una campaña real completa.

3.5.1. Interferencia del propio instrumento sobre la medición que reemplaza

Dado que los contadores de *uncore* son de alcance de sistema (sección 1.1.3), habilitarlos exige reserva exclusiva del nodo — verificada como precondition bloqueante antes de iniciar cualquier campaña que los solicite, para que ningún proceso ajeno pueda contaminar una lectura que, por diseño, no puede atribuirse a un proceso concreto. Satisfecha esa condición, antes de comprometer una campaña real de varias horas al instrumento ya validado en la sección anterior, se aplicó una segunda disciplina: una prueba de humo, a escala reducida y de minutos de duración, que ejercita el mismo camino de código de principio a fin — verificación previa, calibración, escritura de frecuencia, *uncore*, post-procesamiento y validación de ventanas — sobre una única carga, antes de repetir el mismo procedimiento contra la campaña completa. Esta prueba expuso un defecto real que ninguna de las verificaciones anteriores, centradas en la corrección del formato de datos y en la disponibilidad del propio subproceso,

había podido exponer: con *uncore* habilitado, la proporción de ventanas aceptadas cayó de 67.8 % — la misma carga, sin *uncore*, bajo el resto del instrumento ya validado — a 0 %, sin que el subproceso de *uncore* hubiera dejado de producir datos reales; el problema no estaba en lo que *uncore* medía, sino en su efecto sobre los contadores de núcleo con los que debía coexistir.

La causa raíz se estableció en dos capas independientes, cada una confirmada por separado antes de aceptar la siguiente como explicación completa. La primera: el subproceso de *uncore* hereda la afinidad de CPU del proceso que lo lanza, sin restricción explícita, de modo que el planificador del sistema operativo puede ubicarlo sobre los mismos núcleos donde se miden los contadores por proceso de la carga observada. Confirmado mediante una prueba aislada — la misma carga, con y sin esta restricción de afinidad —: sin restringirla, el evento `FP_ARITH_INST_RETIRED` (sección 2.3.8) quedaba ausente en 66.2 % de las ventanas; anclando el subproceso a un núcleo fuera de los delegados a la carga, la ausencia bajó a 0 %. Corregido fijando la afinidad del subproceso de *uncore* a un núcleo reservado para telemetría, nunca a los núcleos donde corre la carga medida.

Esa corrección, sin embargo, no bastó por sí sola: la misma prueba de humo repetida tras aplicarla seguía sin aceptar ninguna combinación. Investigado el mismo evento con la restricción de afinidad ya aplicada, un segundo defecto emergió, esta vez independiente de *uncore* por completo — confirmado repitiendo la misma medición sin habilitarlo en absoluto, con idéntico resultado —: el contador `FP_ARITH_INST_RETIRED` (sub-evento escalar) no aparecía ausente, sino que su valor crudo *decrecía* entre lecturas consecutivas en 57.8–57.9 % de las ventanas, la firma de un contador reprogramado a un registro físico distinto entre lecturas por el mecanismo de multiplexación descrito en la sección 1.1.3. La causa: un mecanismo estándar del sistema operativo, ajeno a este proyecto y presente en cualquier despliegue de servidor típico — la vigilancia por interrupción no enmascarable ante bloqueos irrecuperables del núcleo — reserva de forma permanente un contador de hardware por núcleo, independientemente de si algún proceso lo solicita. El presupuesto de diez contadores simultáneos verificado como sostenible en la sección 2.3.8 nunca descontó esa reserva; solicitar los diez completos forzaba al planificador a rotar uno de ellos entre lecturas para hacerlos caber en el presupuesto físico realmente disponible — una condición ausente en el momento en que aquella verificación se realizó, y presente al momento de esta prueba de humo, sin que exista forma de determinar exactamente cuándo cambió. No es posible desactivar ese mecanismo sin alterar una política de todo el nodo ajena al alcance de este proyecto; en su lugar, se redujo el propio presupuesto solicitado a nueve, descartando el contador de menor valor metodológico entre los diez — el *proxy* de tráfico de caché de segundo nivel, ya sin ningún papel en la

clasificación desde que *uncore* lo reemplazó como fuente admitida, conservado hasta entonces únicamente como columna informativa de contraste.

Verificado el efecto de ambas correcciones en conjunto, primero de forma aislada y luego mediante dos campañas de humo completas: la razón mínima de tiempo contando sobre tiempo habilitado se sostuvo en 1.0 a lo largo de toda una corrida de prueba, sin ningún delta negativo en los cuatro sub-eventos de punto flotante; una campaña de humo sobre la misma carga con barrido completo de frecuencia (REF y F0–F4, dieciocho combinaciones) aceptó la totalidad; y una segunda campaña de humo, esta vez sobre los siete kernels reales del catálogo de conjunto de datos a frecuencia nativa (veintiuna combinaciones), aceptó también la totalidad, con clasificaciones físicamente coherentes: la multiplicación densa de matrices resultó *compute-bound* en el 100 % de sus ventanas válidas, con intensidad operacional entre 15.0 y 29.6 FLOP/byte, muy por encima del punto de inflexión corregido (sección 3.6); los seis kernels de la suite NAS Parallel Benchmarks resultaron predominantemente *memory-bound*, con la excepción de una fracción minoritaria de ventanas *compute-bound* en el kernel de resolución de sistemas por métodos implícitos, consistente con el desplazamiento del punto de inflexión documentado en esa misma sección. Solo tras esta doble verificación — la corrección aplicada y su efecto confirmado a la escala completa del catálogo, no solo sobre la carga que originalmente expuso el defecto — se dio por lista la instrumentación para sostener la campaña real completa (sección 3.2).

3.6. Validación cruzada de los techos Roofline con una herramienta independiente

Los techos del modelo Roofline se determinan, como se explicó en la sección 1.1.2, mediante calibración empírica sobre la propia plataforma. Esa calibración depende, a su vez, de que los microbenchmarks de calibración alcancen efectivamente el máximo real de la plataforma: un microbenchmark que subestima el techo produce un punto de inflexión desplazado, y con él, una clasificación sistemáticamente sesgada de toda ventana cuya intensidad caiga en la región afectada por ese desplazamiento — sin que el instrumento de medición por ventana, correcto en sí mismo, pueda detectarlo. Verificar el resultado de la propia calibración exige, por tanto, una vía independiente del mecanismo de calibración: se empleó Intel Advisor, una herramienta de perfilado que construye su propio análisis Roofline mediante instrumentación binaria directa del bucle más costoso de un programa — contabilizando FLOPs y bytes ejecutados realmente, sin pasar por contadores de PMU en absoluto —, un mecanismo de medición genuinamente distinto al empleado por este instrumento, y por tanto una validación

cruzada real y no una repetición del mismo método [40].

Contrastado sobre una multiplicación de matrices densas de referencia (biblioteca de álgebra lineal optimizada, con vectorización AVX-512 confirmada), Advisor midió 577.7 GFLOP/s con una intensidad operacional de 1.091 FLOP/byte sobre los mismos seis núcleos de la calibración de producción — coherente con el punto de inflexión propio del catálogo (sección 1.1.2). El ancho de banda calibrado por este instrumento validó adecuadamente contra Advisor: 58.4–58.8 GB/s frente a 64.0–66.8 GB/s medidos de forma independiente sobre el mismo microbenchmark, una diferencia de 12–15 %, dentro de lo esperable entre dos mecanismos de medición distintos del mismo fenómeno físico. El techo de cómputo, en cambio, no validó: el microbenchmark propio de densidad aritmética medía apenas 71.8–71.9 GFLOP/s, una brecha de casi $8\times$ frente a la referencia de Advisor — demasiado grande para atribuirse a una diferencia de metodología entre dos herramientas, y consistente en cambio con un defecto real del propio microbenchmark de calibración.

Inspeccionado el bucle interno de ese microbenchmark con el mismo reporte de Advisor, la causa quedó identificada de forma directa: el conjunto de instrucciones vectoriales realmente generado por el compilador era SSE2 — la base garantizada del conjunto de instrucciones x86-64, de ancho 128 bits — pese a que la plataforma soporta AVX-512 (ancho 512 bits, ocho elementos de doble precisión por instrucción, sección 2.3.8). La causa raíz fue de compilación, no de diseño del microbenchmark: la invocación del compilador no solicitaba explícitamente el conjunto de instrucciones nativo de la plataforma, y sin esa solicitud explícita el compilador nunca genera instrucciones más anchas que la base garantizada del conjunto de instrucciones — una inconsistencia real frente al resto del instrumento, que sí solicita explícitamente el conjunto de instrucciones nativo en su propia cadena de compilación. Corregida esa invocación, el rendimiento medido subió a 308.5 GFLOP/s ($4.3\times$), pero la inspección con Advisor mostró que el compilador, incluso solicitando el conjunto de instrucciones nativo, había optado por generar AVX2 (ancho 256 bits) en vez de AVX-512 — una heurística deliberada del propio compilador en objetivos de servidor, que evita por defecto el ancho vectorial máximo porque su uso sostenido puede inducir una reducción de la frecuencia de reloj del núcleo. Solicitado explícitamente el ancho vectorial máximo, el rendimiento medido subió una segunda vez, a 417.8–555 GFLOP/s.

Esa misma bandera de ancho vectorial explícito se evaluó también sobre el microbenchmark de ancho de banda, y se descartó para él tras encontrar una regresión real: el ancho de banda medido bajó y se volvió considerablemente más variable entre corridas (49.3–69.2 GB/s, frente a 58.8–59.5 GB/s estable sin la bandera) — consistente con la misma reducción de frecuencia bajo ancho vectorial máximo, sin ningún beneficio de cómputo para

un microbenchmark limitado por ancho de banda de memoria y no por ejecución vectorial. La corrección se aplicó, en consecuencia, de forma selectiva: ancho vectorial máximo únicamente en el microbenchmark de densidad aritmética, nunca en el de ancho de banda. El mismo defecto de compilación — ausencia de solicitud explícita del conjunto de instrucciones nativo — se encontró también en la cadena de construcción de los seis kernels de conjunto de datos de la suite NAS Parallel Benchmarks; corregido igualmente, pero sin la bandera de ancho vectorial máximo, dado que esa suite combina kernels de régimen memory-bound y compute-bound bajo una única configuración de compilación compartida, con el mismo riesgo de regresión ya observado para el microbenchmark de ancho de banda.

El desplazamiento resultante del punto de inflexión es sustancial: de aproximadamente 1.22 FLOP/byte con el defecto, a un valor final en el rango 7.0–9.3 FLOP/byte una vez corregido — una diferencia de magnitud suficiente para invertir la clasificación de toda ventana cuya intensidad caiga entre ambos valores, exactamente el rango donde se concentra buena parte de las cargas de conjunto de datos de este catálogo (sección 3.5.1). Por invariancia del propio modelo Roofline, este desplazamiento no sesga la intensidad operacional observada de ninguna ventana — esa magnitud es una propiedad del algoritmo y es invariante al ancho vectorial con el que se ejecuta, medida directamente por el contador de hardware descrito en la sección 2.3.8 con independencia de cuántos elementos procese cada instrucción —; afecta exclusivamente la posición del techo contra el cual esa intensidad se compara. La primera campaña de CPU se calibró bajo el punto de inflexión defectuoso y quedó descartada por este motivo y por la ausencia de *uncore*; la campaña limpia de reemplazo ya emplea ambos mecanismos corregidos, aunque su eje de frecuencia queda limitado por el hallazgo de turbo descrito en la sección 3.2.

Capítulo 4

Discusión

Los resultados de esta primera fase sostienen dos tipos de contribución distintos: uno metodológico, sobre cómo construir con rigor un instrumento de caracterización DVFS antes de confiar en él; y uno empírico, sobre el comportamiento real de la plataforma de medición frente a ese instrumento.

Buena parte de la literatura revisada en el estado del arte — los métodos de selección de frecuencia óptima de Ali et al. [5] y de clasificación consciente de DVFS de Guerreiro et al. [19], o el marco de control por refuerzo de Wang et al. [26] — asume, de forma razonable dado su alcance, que la escritura de frecuencia sobre el hardware subyacente funciona como el fabricante la documenta, y concentra su aporte en la capa de decisión: qué frecuencia elegir, o cómo aprenderla. Los resultados de actuación muestran que esa suposición debe verificarse empíricamente bajo carga real: en GPU, la prueba positiva sostuvo dos objetivos distintos durante cientos de lecturas activas y confirmó el mecanismo; en CPU, en cambio, la relectura correcta de los límites escritos quedó refutada como prueba de actuación suficiente cuando el nuevo muestreo durante la carga mostró que el turbo global mantenía una frecuencia distinta. Para cualquier política construida sobre estos actuadores, la diferencia entre éxito administrativo, estado solicitado y efecto físico solo aparece al contrastar directamente el hardware durante el trabajo real. La medición de bytes de *uncore* (sección 3.5) confirma el mismo principio por una vía independiente: un permiso comunicado como concedido no coincidió inicialmente con el estado verificable del sistema, y solo la comprobación directa contra el binario real dejó en evidencia la brecha y permitió confirmar su corrección posterior.

En la misma línea, la corrección del chequeo de carga externa (sección 3.2) ilustra un riesgo metodológico general en la validación de instrumentos de medición sobre cargas propias: una verificación de seguridad diseñada correctamente para un escenario de prueba corto

puede fallar sistemáticamente a escala real, precisamente porque a esa escala dispone de tiempo suficiente para que la señal que observa converja hacia un valor indistinguible de una condición de alarma legítima. Este tipo de falla — a diferencia de un error de sintaxis o de una condición de contorno mal definida — no se manifiesta en pruebas unitarias herméticas ni en una campaña breve; solo aparece al ejercitar el instrumento en las condiciones reales para las que fue diseñado, lo que refuerza la elección metodológica de este trabajo de tratar la campaña real como parte del proceso de validación del instrumento, y no solo como su producto final.

El hallazgo de la sección 3.5.1 extiende ese mismo principio en una dirección que las secciones anteriores no habían cubierto todavía: no basta con verificar que cada mecanismo del instrumento funciona por separado — la escritura de frecuencia, la lectura de *uncore*, la medición directa de FLOPs —, porque dos mecanismos correctos en aislamiento pueden interferir entre sí de una forma que ninguna prueba unitaria, centrada por construcción en un solo mecanismo a la vez, puede exponer. El presupuesto de contadores simultáneos verificado como sostenible en la sección 2.3.8 no era, en sentido estricto, incorrecto en el momento en que se verificó; dejó de sostenerse cuando cambió una condición del entorno — una reserva de contador realizada por un mecanismo del propio sistema operativo, ajeno tanto al instrumento como al proyecto — que ninguna verificación anterior tenía motivo para volver a comprobar. La lección metodológica no es que el presupuesto físico de contadores deba tratarse como una constante de la plataforma, sino que ninguna capacidad de hardware nominal — por verificada que haya estado en un momento dado — puede asumirse estable en el tiempo sin volver a contrastarla contra el estado real del sistema; es la misma disciplina de verificación de las secciones 1.1.5 y 1.1.3, aplicada aquí no a la existencia de un permiso, sino a su persistencia.

Esa misma sección ilustra, además, el valor de una práctica adoptada explícitamente en la segunda mitad de este trabajo: antes de comprometer una campaña real de varias horas, ejecutar una réplica reducida — de minutos, sobre una fracción del catálogo o de la matriz de frecuencias — que ejercite el mismo camino de código de principio a fin. El defecto que esa práctica expuso no era detectable por inspección de código ni por las pruebas unitarias ya existentes, que verifican el formato de los datos y la degradación controlada del instrumento, pero no su interacción con el estado dinámico del sistema operativo bajo carga real; y de no haberse expuesto en una prueba de minutos, se habría descubierto — en el mejor de los casos — al final de una campaña de horas, con el consiguiente costo de cómputo perdido, o — en el peor — habría contaminado silenciosamente el conjunto de datos resultante, dado que una ventana con contadores degradados se descarta de la clasificación pero no impide

que la corrida se acepte si el resto de sus ventanas son suficientes. Generalizar esta práctica — una prueba de humo a escala reducida como paso obligatorio antes de cualquier campaña de producción, no solo la primera — es, junto con la verificación empírica ya discutida, un segundo aporte metodológico de esta fase.

La validación cruzada de los techos Roofline (sección 3.6) constituye una tercera instancia del mismo principio, aplicada esta vez a la integridad de la propia calibración y no a un permiso o a un mecanismo de actuación. Su hallazgo comparte una estructura común con los dos anteriores: un componente que producía un resultado plausible — un microbenchmark de calibración que compilaba, corría y reportaba una cifra de rendimiento sin ningún error visible — resultó, contrastado contra un mecanismo de medición independiente, sistemáticamente incorrecto por casi un orden de magnitud, sin que ninguna verificación interna del propio instrumento pudiera haberlo detectado: la ausencia de error no es evidencia de corrección cuando el propio mecanismo de verificación comparte la misma fuente de sesgo que el componente que pretende verificar. La elección deliberada de aplicar la corrección resultante — el ancho vectorial explícito de compilación — de forma selectiva y no uniforme entre microbenchmarks, tras encontrar que beneficiaba a uno y perjudicaba a otro, refuerza además un punto ya presente en la discusión de la sección 3.3: una corrección que soluciona un defecto real en un caso concreto no debe generalizarse sin verificar, caso por caso, que el mecanismo físico que la justifica aplica igual en cada uno.

El hallazgo de precisión mixta en el catálogo de GPU (sección 3.3) es relevante más allá de este trabajo concreto. La práctica de derivar la precisión de caracterización de un kernel a partir de su tipo de dato declarado en el código fuente — en lugar de la precisión efectivamente ejercitada por el hardware durante su ejecución — es razonable como primera aproximación, pero este trabajo encontró que puede diferir de lo medido en más del 70 % del trabajo aritmético de un kernel, y que esa diferencia puede situarse exactamente en el margen que decide una clasificación Roofline. Ningún trabajo de los revisados en el estado del arte reporta explícitamente esta verificación para GPU, lo que sugiere que la práctica de asumir la precisión declarada, sin contrastarla, podría no ser inusual en caracterizaciones similares.

Finalmente, el resultado sobre precisión de `gpu_dgemv_n4096` (sección 3.3) matiza — sin contradecir — el hallazgo de Guerreiro et al. sobre la sensibilidad de las cargas GPGPU a la ruta de ejecución elegida por bibliotecas de terceros [19]: una ruta acelerada por hardware especializado invalida efectivamente un método de caracterización diseñado para instrucciones aritméticas convencionales, pero no invalida necesariamente la magnitud medida, siempre que exista una vía de verificación independiente — en este caso, el conteo analítico del propio

algoritmo — contra la cual contrastarla.

Capítulo 5

Conclusiones

Este trabajo completó la construcción y validación empírica del instrumento de caracterización CPU–GPU de la Fase 1, pero el conjunto DVFS final aún requiere cerrar los dos recorridos experimentales pendientes. Respecto al primer objetivo específico — caracterizar cargas representativas bajo distintos estados de frecuencia —, el resultado es parcial: en CPU, la campaña limpia de reemplazo terminó con 126/126 combinaciones aceptadas e incorpora FLOPs por hardware, bytes reales de *uncore* y el ridge corregido, pero el muestreo posterior de frecuencia efectiva demostró que F0–F4 no constituyeron estados físicos distintos mientras el turbo global permaneció activo; por ello esos datos no sustentan todavía comparaciones DVFS de tiempo, energía o EDP. En GPU, el catálogo quedó caracterizado y auditado por precisión, el instrumento multi-frecuencia está implementado y `-lgc` ya retiene el reloj bajo carga real, pero falta ejecutar y aceptar la campaña completa. Los objetivos específicos restantes — entrenamiento del clasificador, implementación del agente y evaluación de EDP — corresponden a las Fases 2–4 y no se presentan como ejecutados.

El aporte metodológico central de esta fase, más allá de los datos concretos recolectados, es haber demostrado — y no solo enunciado — que ningún mecanismo de este instrumento se dio por funcional sin verificación directa contra el estado real del hardware: ni la disponibilidad de un contador de PMU, ni el presupuesto físico de contadores simultáneos, ni su persistencia en el tiempo frente a mecanismos ajenos al instrumento, ni la escritura de frecuencia de CPU, ni la precisión aritmética declarada de un kernel de GPU, ni la calibración de los propios techos del modelo Roofline. En varias ocasiones documentadas en este trabajo, esa verificación expuso una falla real que una confianza no verificada en la documentación del fabricante, en una medición previa ya dada por válida, o en el diseño original del instrumento, habría dejado pasar silenciosamente hacia el conjunto de datos.

5.1. Limitaciones

Este trabajo reconoce las siguientes limitaciones. Primera, aunque la clasificación CPU depende exclusivamente de bytes medidos mediante *uncore*, esa señal tiene un piso práctico de intervalo de aproximadamente 10 ms, más grueso que las ventanas de CPU de aproximadamente 1 ms; la intensidad y la etiqueta son, por tanto, propiedades del intervalo *uncore* difundidas a las ventanas que cubre, no mediciones independientes a 1 ms. Segunda, el presupuesto simultáneo de PMU depende del estado de mecanismos del sistema operativo ajenos al proyecto; se redujo a nueve eventos para convivir con la reserva encontrada, pero una reserva futura adicional exigiría repetir la verificación. Tercera, el control CPU por núcleo no domina el turbo global en la configuración actual: el lector por ventana permite detectar la divergencia, pero resolverla requiere autoridad administrativa sobre *no_turbo* o *max_perf_pct*; la temperatura de paquete tampoco está conectada a un gate bloqueante de sensor. Cuarta, el catálogo y los *encodings* crudos están restringidos a la microarquitectura de la Tabla 2.1; no se reclama portabilidad de los mecanismos de actuación sin repetir sus verificaciones bajo carga en cada estado de plataforma. Quinta, aunque el actuador GPU ya fue validado en dos frecuencias bajo carga, todavía no existe una campaña GPU multi-frecuencia completa aceptada, y la campaña CPU disponible no contiene niveles efectivos diferenciados.

5.2. Trabajo Futuro

En el corto plazo, tres tareas siguen directamente habilitadas. Primera, resolver con la administración el control del turbo global de CPU y repetir la campaña limpia, usando el lector por ventana para aceptar cada nivel por su frecuencia efectiva y no solo por la relectura de sysfs. Segunda, ejecutar la campaña GPU multi-frecuencia ahora que *-lgc* fue verificado bajo carga, comenzando por una prueba de humo que ejercite el mismo producto cartesiano y los mismos gates de la campaña final. Tercera, consolidar ambos conjuntos y definir la partición experimental de la Fase 2 por corrida o carga, evitando tratar como observaciones independientes ventanas que comparten una etiqueta de intervalo *uncore* o una intensidad GPU caracterizada por kernel.

Bibliografía

- [1] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba, “Increasing Energy Efficiency of Astrophysics Simulations Through GPU Frequency Scaling,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1826–1834. doi: 10.1109/SCW63240.2024.00229.
- [2] M. T. Costa et al., “Characterizing the Impact of GPU Power Management on an Exascale System,” in *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, New York, NY, USA, 2025, pp. 1524–1533. doi: 10.1145/3731599.3767702.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama, “MCBound: An Online Framework to Characterize and Classify Memory/Compute-bound HPC Jobs,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1–15. doi: 10.1109/SC41406.2024.00062.
- [4] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione, “Evaluation of DVFS techniques on modern HPC processors and accelerators for energy-aware applications,” *Concurrency Comput. Pract. Exper.*, vol. 29, no. 12, p. e4143, 2017. doi: 10.1002/cpe.4143.
- [5] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, “An automated and portable method for selecting an optimal GPU frequency,” *Future Generation Computer Systems*, vol. 149, pp. 71–88, Dec. 2023. doi: 10.1016/j.future.2023.07.011.
- [6] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, “Supply and threshold voltage scaling for low power CMOS,” *IEEE J. Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, Aug. 1997. doi: 10.1109/4.604018.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009. doi: 10.1145/1498765.1498785.

- [8] T.-Y. Liu, J. Guo, and B. Huang, “Efficient Cross-Platform Multiplexing of Hardware Performance Counters via Adaptive Grouping,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 1, pp. 1–26, Mar. 2024. doi: 10.1145/3629525.
- [9] S.-K. Shekoffteh et al., “Metric selection for GPU kernel classification,” *ACM Trans. Archit. Code Optim.*, vol. 15, no. 4, pp. 1–27, Jan. 2019. doi: 10.1145/3295690.
- [10] L. Littman and T. Deakin, “Classifying Performance Bounds Using Machine Learning,” in *Proc. SC25: Int. Conf. High Perform. Comput. Netw. Storage Anal.*, 2025.
- [11] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2020.
- [12] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.
- [13] S. McCarty, “Architecting Containers Part 1: Why Understanding User Space vs. Kernel Space Matters,” Red Hat Blog, Jul. 29, 2015. [Online]. Available: <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters> [Accessed: Apr. 3, 2025].
- [14] V. M. Weaver, “Self-monitoring overhead of the Linux perf event performance counter interface,” in *Proc. IEEE Int. Symp. Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 102–111.
- [15] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, “RAPL: Memory power estimation and capping,” in *Proc. 16th ACM/IEEE Int. Symp. Low Power Electron. Des. (ISLPED)*, Aug. 2010, pp. 189–194. doi: 10.1145/1840845.1840883.
- [16] NVIDIA, “NVIDIA Management Library (NVML),” NVIDIA Developer Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api/index.html>. [Accessed: Apr. 3, 2026].
- [17] P. Thamm and U. Leser, “Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters,” arXiv preprint arXiv:2505.09375, 2025.
- [18] R. Hebbar and A. Milenković, “PMU-events-driven DVFS techniques for improving energy efficiency of modern processors,” *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 7, no. 1, pp. 1–31, May 2022. doi: 10.1145/3538645.

- [19] J. Guerreiro, N. Roma, P. Tomás, F. Pratas, L. S. G. Carvalho, and G. Gaydadjiev, “DVFS-aware application classification to improve GPGPUs energy efficiency,” *Parallel Comput.*, vol. 83, pp. 93–117, May 2019. doi: 10.1016/j.parco.2018.02.001.
- [20] ScienceDirect, “Computational Kernel,” Computer Science Topics, 2026. [Online]. Available: <https://www.sciencedirect.com/topics/computer-science/computational-kernel>. [Accessed: Apr. 3, 2026].
- [21] D. Pandey, K. Niwaria, and B. Chourasia, “Machine Learning Algorithms: A Review,” *Int. J. Mach. Learn. Netw. Appl.*, vol. 6, no. 2, pp. 916–922, Jan. 2019. doi: 10.21275/ART20203995.
- [22] IBM, “What is random forest?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/random-forest>. [Accessed: Apr. 3, 2026].
- [23] IBM, “What is logistic regression?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/logistic-regression>. [Accessed: Apr. 3, 2026].
- [24] IBM, “What is XGBoost?” IBM Think Topics, 2026. [Online]. Available: <https://www.ibm.com/think/topics/xgboost>. [Accessed: Apr. 3, 2026].
- [25] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan, “Energy Delay Product,” in *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science. London, UK: Springer, 2013, ch. 6, pp. 51–55. doi: 10.1007/978-1-4471-4492-2_9.
- [26] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, “DRLCap: Runtime GPU Frequency Capping with Deep Reinforcement Learning,” *IEEE Trans. Sustain. Comput.*, vol. 9, no. 5, pp. 712–726, Sept.–Oct. 2024. doi: 10.1109/TSUSC.2024.3361596.
- [27] D. Velicka, O. Vysocky, and L. Riha, “Methodology for GPU frequency switching latency measurement,” in *Proc. 2025 IEEE Int. Parallel Distrib. Process. Symp. Workshops (IPDPSW)*, 2025, pp. 830–839. doi: 10.1109/IPDPSW66978.2025.00133.
- [28] Z. Zhang, Z. Wu, J. Liu, and L. Mottola, “SparseDVFS: Sparse-Aware DVFS for Energy-Efficient Edge Inference,” arXiv preprint arXiv:2603.21908, Mar. 23, 2026.
- [29] J. Treibig, G. Hager, and G. Wellein, “LIKWID: A Lightweight Performance-Oriented Tool Suite for x86 Multicore Environments,” in *Proc. 39th Int. Conf. Parallel Processing Workshops (ICPPW)*, San Diego, CA, USA, 2010, pp. 207–216. doi: 10.1109/ICPPW.2010.38.

- [30] G. Hamerly, E. Perelman, J. Sherwood, and B. Calder, “SimPoint 3.0: Faster and More Flexible Program Phase Analysis,” *Journal of Instruction-Level Parallelism*, vol. 7, pp. 1–28, 2005.
- [31] Intel, “CPU Metrics Reference,” Intel VTune Profiler User Guide, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2026-1/cpu-metrics-reference.html>. [Accessed: Aug. 7, 2026].
- [32] Innovative Computing Laboratory, University of Tennessee, “PAPI: Performance Application Programming Interface,” 2026. [Online]. Available: <https://icl.utk.edu/papi/>. [Accessed: Aug. 7, 2026].
- [33] Intel Corporation, “perfmon: Performance Monitoring Events for Intel Architecture Processors” (evento FP_ARITH_INST_RETIRED, familia Ice Lake), 2026. [Online]. Available: <https://github.com/intel/perfmon>. [Accessed: Aug. 10, 2026].
- [34] National Energy Research Scientific Computing Center (NERSC), “Roofline Performance Model,” NERSC Documentation, 2026. [Online]. Available: <https://docs.nersc.gov/tools/performance/roofline/>. [Accessed: Aug. 7, 2026].
- [35] V. M. Weaver and S. A. McKee, “Can hardware performance counters be trusted?,” in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Seattle, WA, USA, 2008, pp. 141–150. doi: 10.1109/IISWC.2008.4636099.
- [36] M. A. Laurenzano, M. M. Tikir, L. Carrington, and A. Snaveley, “PEBIL: Efficient static binary instrumentation for Linux,” in *Proc. IEEE Int. Symp. Performance Analysis of Systems & Software (ISPASS)*, White Plains, NY, USA, 2010, pp. 175–183. doi: 10.1109/ISPASS.2010.5452024.
- [37] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd Annu. ACM SIGPLAN Conf. Object-Oriented Program. Syst. Appl. (OOPSLA ’07)*, Montreal, QC, Canada, 2007, pp. 57–76. doi: 10.1145/1297027.1297033.
- [38] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, “Virtual Machine Warmup Blows Hot and Cold,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, art. 52, pp. 1–27, Oct. 2017. doi: 10.1145/3133876.
- [39] NVIDIA, “Nsight Compute,” NVIDIA Developer Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/nsight-compute/>. [Accessed: Aug. 8, 2026].

- [40] Intel Corporation, “Roofline Analysis with Intel Advisor,” Intel Advisor User Guide, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html>. [Accessed: Aug. 12, 2026].
- [41] Intel Corporation, “Intel® 64 and IA-32 Architectures Software Developer’s Manual, Volume 3B: System Programming Guide, Part 2” (Performance Monitoring, NMI watchdog interaction with fixed-function and general-purpose counters), 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. [Accessed: Aug. 12, 2026].

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>